Credit Card Fraud Detection with Machine Learning

Exploratory Analysis, Preprocessing, Model Selection, Evaluation, and Responsible AI

Table of Contents

Section	Page
Project Overview	2
Exploratory Data Analysis	6
Data Preprocessing	15
Model Development and Selection	24
Model Evaluation and Visualisation	38
Responsible AI, Ethics, and Governance	47
Overall Conclusion	51
References	52
Appendix A: Cloud Deployment Feasibility	55

Technical Portfolio Report

Project Overview

Section Contents

1 Project Overview **4**

1 Project Overview

Credit-card fraud detection is a binary supervised-learning problem in which each transaction is classified as either legitimate (`Class = 0`) or fraudulent (`Class = 1`). The objective is not simply to maximise overall accuracy, because legitimate transactions dominate the dataset and a majority-class classifier could appear accurate while failing to identify the rare cases of greatest operational importance. The report therefore treats fraud detection as an imbalance-aware decision problem in which missed fraud and unnecessary alerts must both be considered.

The analysis uses `creditcard.csv`, which contains **284,807 transactions** and **31 variables**. Thirty variables are predictors, comprising `Time`, `Amount`, and the anonymised components `V1-V28`; `Class` is the binary target. The data is fully numerical and contains no missing values. Fraudulent transactions represent approximately **0.17%** of the observations, compared with **99.83%** legitimate transactions. This severe imbalance makes accuracy an insufficient measure and motivates the use of precision, recall, F1-score, ROC-AUC, Precision–Recall analysis, and confusion matrices.

Summary

The technical workflow proceeds through five connected stages:

1. exploratory analysis of data quality, class distribution, outliers, and feature relationships;
2. leakage-controlled preprocessing and comparison of imbalance-handling strategies;
3. broad model screening, cross-validation, and hyperparameter tuning;
4. final evaluation of the strongest models using scalar and visual evidence; and
5. responsible-AI assessment covering fairness, privacy, explainability, governance, oversight, and sustainable cloud use.

The preprocessing workflow was designed to preserve the realism of the final evaluation. No imputation or categorical encoding was required. IQR-based filtering was explored to understand the highly skewed `Amount` distribution, but extreme observations were not removed from the final modelling data because unusual transactions may contain relevant fraud signal. Features and target were separated before a stratified train–test split, and scaling was fitted only on the training data to prevent leakage. `StandardScaler` was retained as the common scaling strategy. Random under-sampling, random over-sampling, SMOTE, and class-weighted learning were examined as alternatives, but the strongest overall configurations ultimately used the original scaled training data.

The model-development stage evaluated Logistic Regression, Decision Tree, Random Forest, XGBoost, K-Nearest Neighbours, and neural-network approaches using both `MLPClassifier` and Keras/TensorFlow. Support Vector Machine was considered during screening but was not retained because of its computational cost on the full dataset. The comparison progressed from an initial test-split ranking to five-fold stratified cross-validation, RandomizedSearchCV tuning, and supplementary threshold analysis. This staged process reduced the risk of selecting a model from a single favourable split or from one isolated metric.

False negatives represent fraudulent transactions that escape detection, while false positives represent legitimate transactions incorrectly flagged for review or intervention. Both error types have operational consequences: missed fraud can create direct financial loss, whereas excessive false positives can increase review workload, interrupt legitimate payments, and damage customer trust. Random Forest and XGBoost, both trained on the original scaled dataset and tuned with RandomizedSearchCV, formed the final shortlist. Random Forest offered slightly stronger precision and fewer false alarms, while XGBoost achieved slightly stronger recall, F1-score, ROC-AUC, and Average Precision.

Because missed fraud is especially costly, the recommended configuration is **XGBoost trained on the original scaled dataset, tuned with RandomizedSearchCV, and used with the retained default decision threshold of 0.5**.

The recommendation is not treated as a purely predictive decision. Responsible deployment requires documented limitations, threshold review, human oversight, monitoring, periodic revalidation, secure data governance, and attention to bias and customer impact. The anonymised predictors also restrict direct business interpretation, so feature importance should support rather than replace domain review and governance controls. Cloud deployment considerations are discussed separately in Appendix A, where AWS, Azure, and GCP are compared and Azure is recommended for the regulated deployment scenario considered in the report.

This portfolio edition is adapted from a collaborative MSc project and consolidates the technical workflow, modelling analysis, evaluation, and report into a standalone professional document. Contextual cloud and responsible-AI material developed within the wider project has been retained where relevant.

The remainder of the report presents the exploratory analysis, preprocessing decisions, model development and selection, final evaluation and visualisation, and responsible-AI analysis. An overall conclusion then consolidates the main findings before the references, followed by the complete cloud deployment feasibility study in Appendix A.

Technical Portfolio Report

Exploratory Data Analysis

Section Contents

1 Introduction **8**

2 Dataset Overview and Data Quality **8**

3 Class Distribution and Imbalance **9**

4 Feature Distributions, Outliers, and Anomalous Behaviour **10**

5 Correlation Analysis and Relationship with the Target Variable **11**

6 Patterns, Anomalies, and Opportunity Identification **12**
6.1 Links to Industry and Research Examples 13

7 Conclusion **14**

1 Introduction

Fraud detection is a high-impact application of machine learning because even relatively small improvements in identifying fraudulent transactions can reduce financial loss, improve customer security, and support more effective operational decision-making. The dataset used in this project (`creditcard.csv`) contains labelled transaction records, where each observation is classified as either *fraudulent (1)* or *legitimate (0)*. This makes the problem a **binary supervised classification task**. This EDA focuses on:

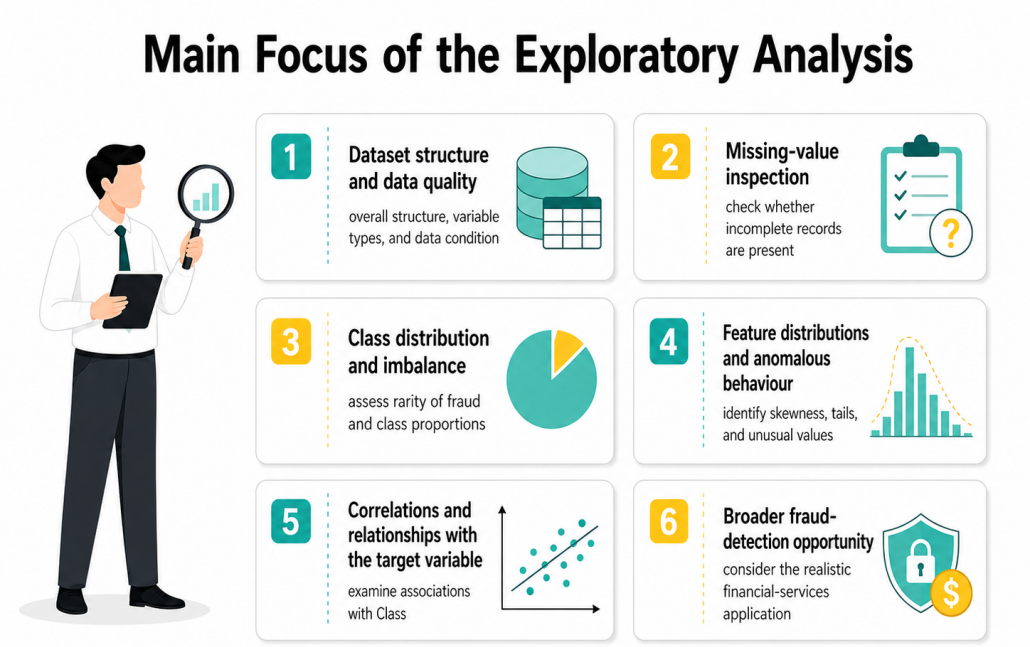


Figure 1: Scope of the exploratory data analysis.

The purpose of the exploratory analysis is not to clean, transform, or model the dataset yet, but to understand its structure, behaviour, and analytical characteristics well enough to identify the main **patterns**, **anomalies**, and **opportunities** it presents.

2 Dataset Overview and Data Quality

The dataset contains **284,807 transactions** and **31 variables**. Of these, `Class` is the target variable, while the remaining variables are predictors. The target is binary:

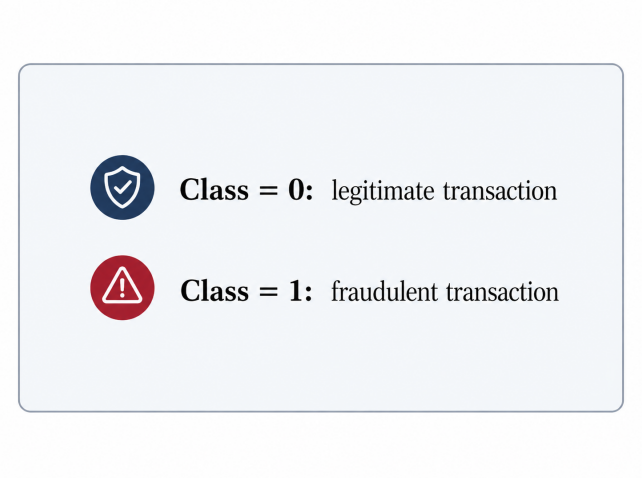


Figure 2: Binary target (`Class`).

Most predictors are already numerical and anonymised, mainly through variables labelled `V1` to `V28`. In addition, `Time` and `Amount` remain in their original form. This has two immediate analytical consequences. First, the dataset is already suitable for standard machine-learning analysis because it is fully numerical. Second, the anonymised feature structure limits direct business interpretation, meaning that useful predictive patterns may be identifiable statistically without always being easily interpretable in operational terms. This is common in fraud data, where privacy and security constraints often limit feature transparency.

Property	Description
Number of observations	284,807 transactions
Number of variables	31 columns
Predictor variables	30 features (`Time`, `Amount`, `V1-V28`)
Target variable	`Class`
Problem type	Binary classification
Data type profile	Fully numerical
Missing values	None detected
Encoding relevance	No categorical feature encoding required

Table 1: Dataset summary.

The dataset inspection showed that **no missing values are present**. This is important because it means that the exploratory analysis is not distorted by incomplete observations. In addition, because all features are already numerical, **no categorical feature encoding is required**. This reduces complexity at the analytical stage and allows the exploratory work to focus directly on more meaningful fraud-detection characteristics such as imbalance, outliers, and feature relationships. From an opportunity-identification perspective, this is already a positive result. Because the dataset is labelled, large, and structurally clean, it provides a realistic basis for developing a fraud-detection system that could support transaction screening, alert prioritisation, or risk scoring in a financial-services environment.

3 Class Distribution and Imbalance

One of the most important characteristics of the dataset is its **extreme class imbalance**. This is important because some classifiers could achieve very high accuracy simply by predicting the majority class most of the

time. In a fraud-detection setting, such a model would be operationally weak because it would miss many of the minority fraud cases that matter most.

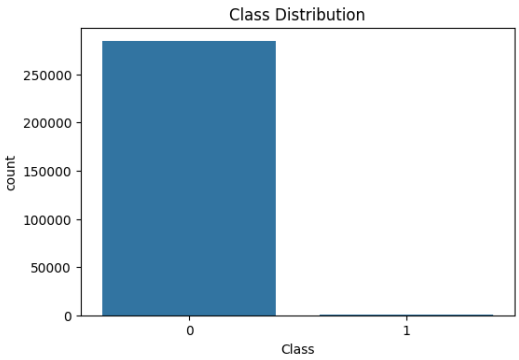
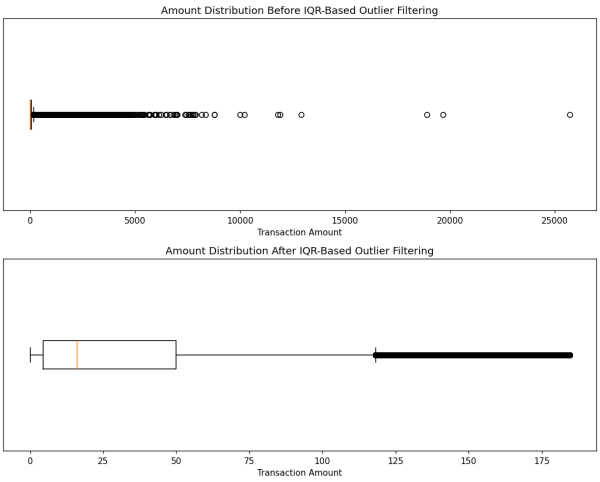


Figure 3: Class distribution of legitimate (99.83%) and fraudulent transactions (0.17%).

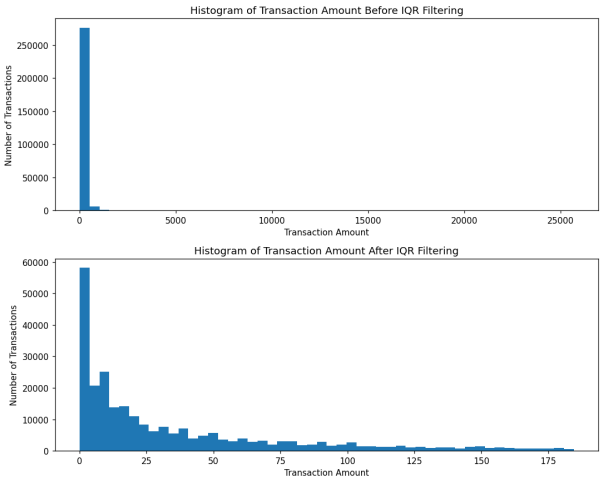
This figure clearly shows the class imbalance. Legitimate transactions dominate the dataset, while fraudulent transactions remain extremely rare. In practice, this means that fraud detection is challenging because the cases of interest are rare, costly, and difficult to identify using simple majority-class predictions. This imbalance therefore represents both a challenge and an opportunity. The challenge is that overall accuracy can become misleading in such a setting. The opportunity is that the minority fraud class is still labelled, meaning that machine learning can potentially learn structured differences between fraudulent and legitimate behaviour rather than relying only on manual rules.

4 Feature Distributions, Outliers, and Anomalous Behaviour

Using the **IQR method**, a large number of high-value outliers were identified in `Amount` (as it is one of the few directly interpretable attributes in the dataset). From a purely statistical perspective, these observations lie far beyond the central range of the data. However, in a fraud-detection context, they should not be dismissed automatically as noise. Unusual transaction amounts may be highly informative and may reflect suspicious behaviour rather than data error. To explore this more clearly, the `Amount` feature was visualised before and after IQR-based filtering using both boxplots and histograms.



(a) `Amount` boxplots before and after IQR filtering.



(b) `Amount` histograms before and after IQR filtering.

Figure 4: Visual comparison of the `Amount` distribution before and after IQR-based filtering.

Figure 4(a) shows that the original `Amount` distribution is heavily affected by extreme values. In the original data, the outliers compress the central part of the distribution and make the main spread harder to inspect. After IQR-based filtering, the central range becomes much clearer, confirming that the original feature contains substantial tail behaviour. Figure 4(b) supports the same conclusion through histograms. The original histogram is strongly concentrated near the lower range because a relatively small number of extreme values stretches the distribution considerably. By contrast, the IQR-filtered version makes the main body of transaction amounts easier to observe. Together, these plots confirm that `Amount` is heavily right-skewed and contains many unusually large values. Although a higher transaction value does not by itself imply a higher probability of fraud, anomalous high-value transactions may be especially important from a business perspective because, if fraudulent, they can involve greater potential financial loss. For this reason, such observations may form part of the signal of interest and may help distinguish fraudulent from legitimate transactions.

5 Correlation Analysis and Relationship with the Target Variable

To understand whether the predictors contain potentially useful relationships, a correlation matrix and heatmap were examined.

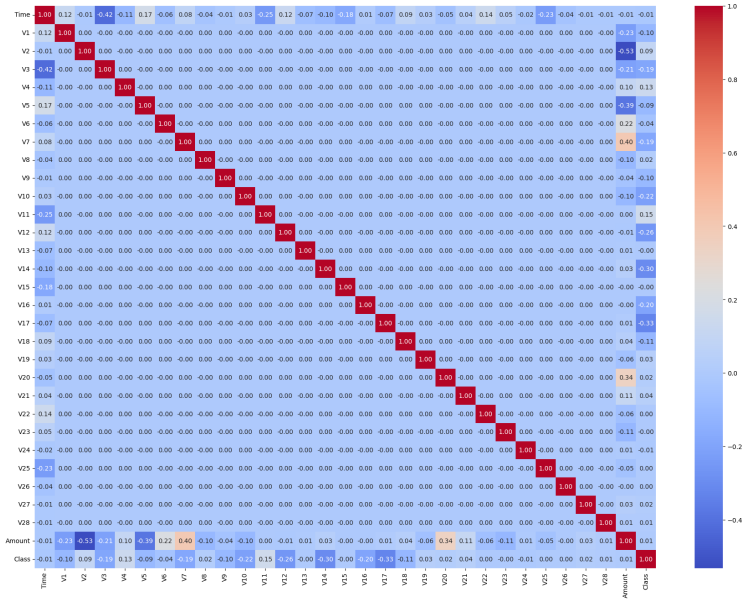


Figure 5: Correlation heatmap for the credit-card fraud dataset.

This heatmap shows that most feature pairs have correlations close to zero. This suggests that the dataset does not contain widespread redundancy through simple pairwise linear dependence. This does *not* mean that the dataset lacks predictive structure; rather, it indicates that many relationships are not strongly linear in a simple two-variable sense. In a fraud-detection setting, this is entirely plausible: fraudulent behaviour may emerge through more subtle or more complex combinations of variables rather than through one dominant linear rule.

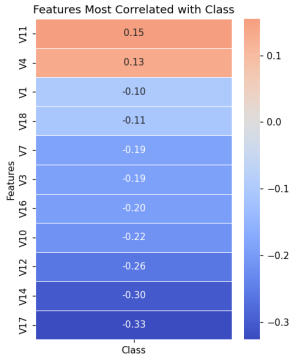


Figure 6: Most relevant correlations with the target variable.

Because the full correlation heatmap is visually dense, a smaller heatmap was also used to focus only on the features whose absolute correlation with `Class` exceeded 0.1. This makes the most relevant relationships easier to interpret directly. When correlations with the target variable `Class` were examined directly, some features showed more noticeable associations with fraud than others. In particular, variables such as `V17`, `V14`, `V12`, and `V10` showed some of the strongest negative relationships with `Class`, while `V11` and `V4` showed comparatively stronger positive relationships. Although these associations remain moderate rather than extreme, they still provide an initial indication that the dataset contains structured signal that a classifier may be able to exploit. This is an important point for opportunity identification. The dataset is not random noise. It contains measurable differences between fraudulent and legitimate behaviour, even if those differences are subtle and distributed across several variables.

6 Patterns, Anomalies, and Opportunity Identification

Taken together, the exploratory analysis suggests that fraudulent and legitimate transactions are not distributed identically across the feature space. This is visible in three main ways:

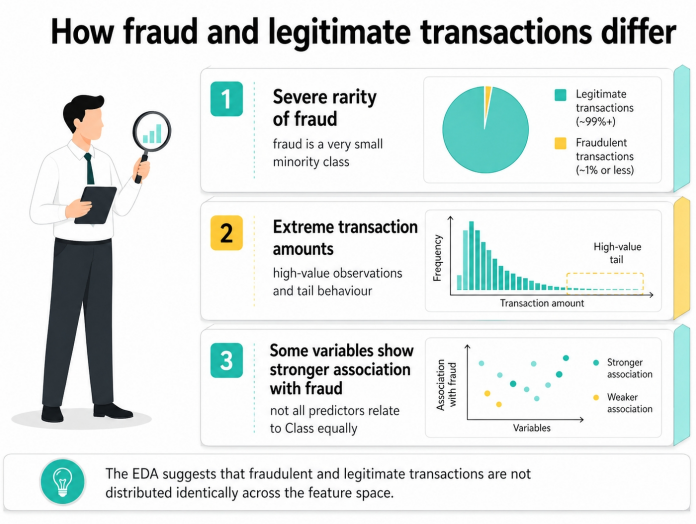


Figure 7: Summary of the main differences between fraudulent and legitimate transactions.

Although the dataset is anonymised, the EDA suggests that the two classes are not completely indistinguishable. The severe class imbalance, the tail behaviour in `Amount`, and the stronger correlations between some variables and `Class` indicate the presence of usable predictive structure. However, these patterns should be interpreted cautiously, as EDA alone does not provide a direct behavioural explanation for fraud. Overall, the dataset

reflects a realistic fraud-detection setting, where fraudulent events are rare, often anomalous, and difficult to detect using simple majority-class rules.

The central opportunity identified through this analysis is the development of a **fraud-risk classification system** to support financial-services decision-making:

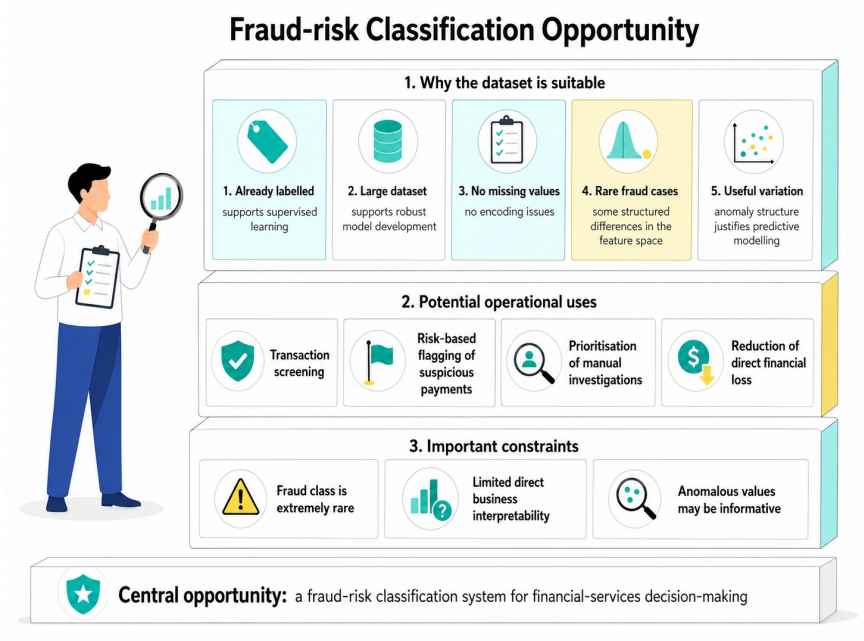


Figure 8: Overview of the fraud-risk classification opportunity.

The figure highlights why the dataset is suitable for supervised modelling, how a fraud-risk system could support financial-services decision-making, and the main constraints and data-handling considerations that should guide development, including rare fraud cases, limited interpretability, and the careful treatment of anomalous values that may contain useful predictive information.

6.1 Links to Industry and Research Examples

The patterns identified in this dataset are consistent with established findings in the fraud-detection literature. Research on credit-card fraud shows that this type of problem is typically characterised by severe class imbalance, subtle and dispersed predictive signals, and the need for evaluation approaches that go beyond simple accuracy measures (Dal Pozzolo et al., 2018; Carcillo et al., 2018). The present EDA reflects these characteristics closely. Fraud accounts for only a very small proportion of the observations, the `Amount` variable shows substantial tail behaviour and anomalous high-value observations, and only a limited number of predictors display noticeable relationships with `Class`, while most pairwise correlations remain weak. This suggests that the dataset captures the kind of fraud-detection setting described in prior studies, where useful predictive information exists but is subtle rather than trivially separable (Dal Pozzolo et al., 2018; Carcillo et al., 2018).

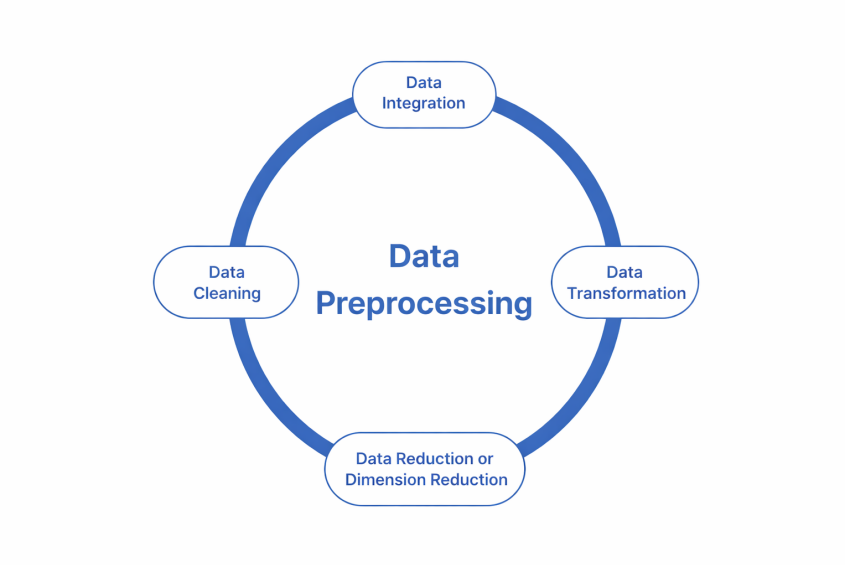
These findings are also relevant from an operational perspective. In real financial-services environments, fraud-detection systems are assessed not only on their ability to identify fraudulent transactions, but also on their capacity to limit false positives. Incorrectly flagging legitimate transactions can damage customer experience, increase investigation costs, and reduce confidence in the system (FICO, 2025). From this perspective, the EDA supports the view that the dataset is suitable for realistic supervised fraud modelling, as it reflects the same challenges found in real-world fraud detection: rarity of fraud, signs of anomalous behaviour, and predictive structure that is present but relatively subtle.

7 Conclusion

The exploratory analysis has shown that the dataset is suitable for supervised fraud modelling, but also analytically challenging. The main findings were the absence of missing values, the fully numerical structure of the data, the severe class imbalance, the anomalous behaviour of `Amount`, and the presence of subtle but usable predictive structure. Taken together, these findings justify moving on to preprocessing and model development in the next stages of the project.

Technical Portfolio Report

Data Preprocessing



Section Contents

1 Introduction **17**

2 Missing-Value Handling and Feature-Encoding Decisions **17**

3 Outlier Handling and the IQR-Based Decision **18**

4 Feature–Target Split and Train–Test Strategy **18**

5 Scaling Strategy **19**

6 Class-Imbalance Handling Strategies **20**
6.1 Why imbalance handling was necessary 20

7 Feature-Engineering Decision **21**

8 Impact of Preprocessing Choices on Downstream Model Performance **21**

9 Conclusion **23**

1 Introduction

The preprocessing stage focuses on preparing the dataset for machine-learning modelling. Building on the exploratory analysis, which established that the `creditcard.csv` dataset is clean, fully numerical, highly imbalanced, and affected by substantial anomalous behaviour, this section translates those findings into a justified preprocessing workflow addressing the following decisions:

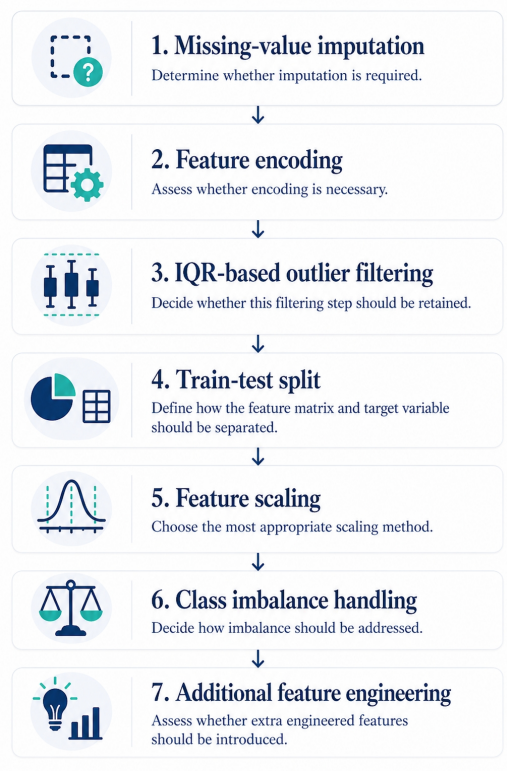


Figure 1: Preprocessing workflow based on the selected decisions.

In fraud detection, preprocessing cannot be treated as a routine technical step. The way missing values, outliers, class imbalance, scaling, and data splitting are handled has a direct impact on later model behaviour and on the realism of the final evaluation. For this reason, the preprocessing stage does not simply apply standard transformations mechanically. Instead, each preprocessing choice is justified in relation to the characteristics of the dataset and to its likely effect on downstream model performance

2 Missing-Value Handling and Feature-Encoding Decisions

The exploratory analysis established that the dataset contains **no missing values** and that all predictors are already **fully numerical**. As a result, no missing-value imputation and no categorical feature encoding were required in the final preprocessing workflow.

Even so, these decisions still needed to be addressed explicitly. In many machine-learning pipelines, missing values are handled through strategies such as row deletion, fixed-value replacement, mean or median imputation, or neighbour-based approaches such as `KNNImputer`.

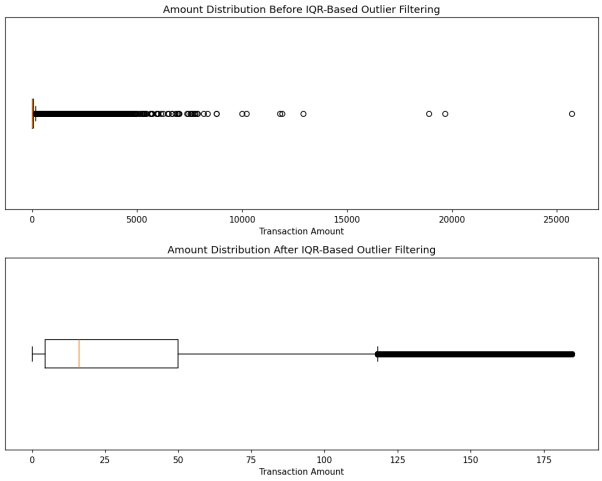
In this project, such methods were considered during preprocessing, but they were not applied in practice because the dataset inspection showed that the number of missing values was zero.

The same applies to feature encoding. In principle, supervised learning pipelines may require label encoding, one-hot encoding, binary encoding, or count/frequency encoding when categorical variables are present. However, the `creditcard.csv` dataset does not contain raw categorical features. The variables `V1` to `V28` are already anonymised and numerical, while `Time` and `Amount` are also numeric. Therefore, encoding was considered as part of the preprocessing decision process, but it was not needed in the final pipeline.

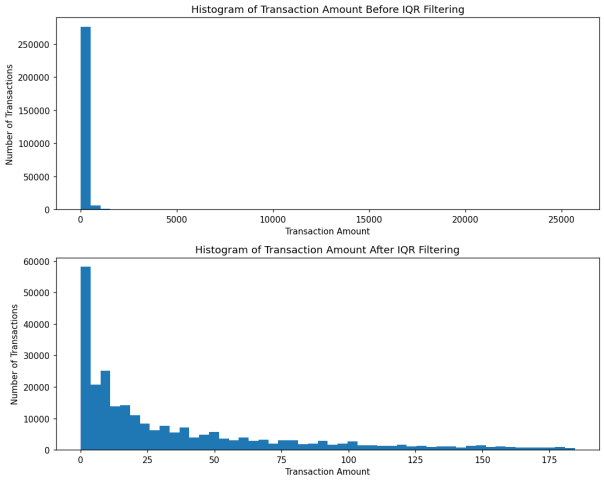
This is an important methodological point. Good preprocessing is not about applying as many techniques as possible. It is about applying only those techniques that are justified by the dataset.

3 Outlier Handling and the IQR-Based Decision

The exploratory analysis showed that the `Amount` feature contains substantial right-skewness and many high-value observations lying outside the IQR-based central range. A filtered version of the dataset was created for exploratory purposes using the IQR rule, and this helped visualise the extent to which extreme values compress the main body of the `Amount` distribution.



(a) `Amount` boxplots before and after IQR filtering.



(b) `Amount` histograms before and after IQR filtering.

Figure 2: Visual comparison of the `Amount` distribution before and after IQR-based filtering.

However, the IQR-filtered dataset was **not retained** in the final preprocessing pipeline. This decision was made because, in a fraud-detection setting, extreme transaction amounts should not automatically be treated as errors or irrelevant noise. Although a higher transaction value does not by itself imply a higher probability of fraud, anomalous high-value transactions may be especially important from a business perspective because, if fraudulent, they can involve greater potential financial loss. For this reason, mechanically removing all extreme values risked discarding observations that may contain useful signal for distinguishing fraudulent from legitimate transactions. The IQR-based filtered dataset was therefore kept only as an **exploratory comparison tool**, not as the basis of the final modelling workflow. This was an important preprocessing decision because it shows that data cleaning must be adapted to the application context. In fraud analytics, some observations that look statistically extreme may in fact be operationally meaningful.

4 Feature–Target Split and Train–Test Strategy

Before any scaling or imbalance handling was applied, the dataset was divided into:

- `X`: the predictor variables,
- `y`: the target variable `Class`.

The next step was a **train–test split** using a stratified 80/20 division. Stratification was necessary because the fraud class is extremely rare, and the split therefore needed to preserve the class proportions as consistently as possible between training and test data.

The split was performed **before** scaling and before any resampling. This was crucial for avoiding data leakage. If scaling or balancing had been applied before the split, information from the future test set could have influenced the training process and made the final evaluation unrealistically optimistic (scikit-learn developers, 2026e,d).

For the same reason, only the training set was used to fit the scaler and to generate resampled versions of the data. The test set remained untouched throughout preprocessing and was treated as unseen data for later evaluation.

5 Scaling Strategy

Two scaling approaches were explored during preprocessing:

- `MinMaxScaler`
- `StandardScaler`

Both methods were considered because the project initially compared several model families, including scale-sensitive classifiers. In principle, scaling matters whenever features operate on different numerical ranges, since this can distort optimisation and distance calculations in models that are not scale-invariant.

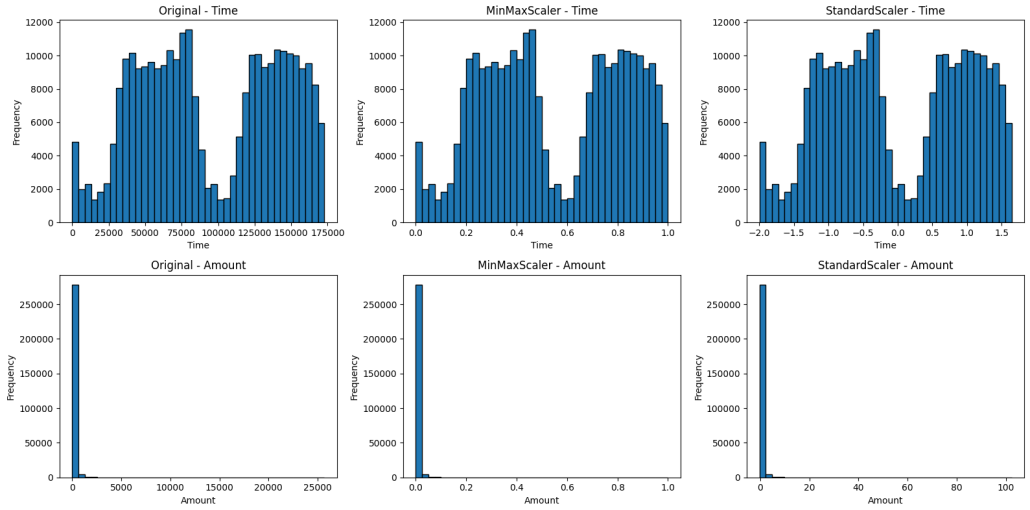


Figure 3: Comparison of the original, Min-Max scaled, and standardised distributions of `Time` and `Amount`.

`MinMaxScaler` was explored as an alternative because it rescales each feature to a common bounded interval, typically between 0 and 1. However, it was not retained as the main scaling strategy in the final pipeline. The main reason is that the dataset contains substantial outlier behaviour, particularly in `Amount`, and Min-Max scaling is highly sensitive to extreme values because it depends directly on the minimum and maximum of each feature. In a distribution with strong tail behaviour, this can compress a large proportion of the observations into a narrow part of the transformed range, making the scaled representation less informative for later modelling. Instead, `StandardScaler` was selected as the main scaling method. Standardisation centres each feature around mean zero and scales it to unit variance, which makes it more suitable for models whose behaviour depends strongly on feature magnitude (scikit-learn developers, 2026f), particularly Logistic Regression, KNN, and Neural Networks. By placing variables on a comparable scale without directly anchoring them to the minimum and maximum values, `StandardScaler` was considered a more robust choice for this dataset.

Although tree-based models such as Decision Tree, Random Forest, and XGBoost do not strictly require scaling, `StandardScaler` was retained across the workflow for consistency of comparison. This allowed the project to evaluate different model families under a common preprocessing setting while still supporting the scale-sensitive classifiers included in the broader modelling stage.

6 Class-Imbalance Handling Strategies

6.1 Why imbalance handling was necessary

The most important preprocessing challenge in this dataset was the severe imbalance between legitimate and fraudulent transactions:

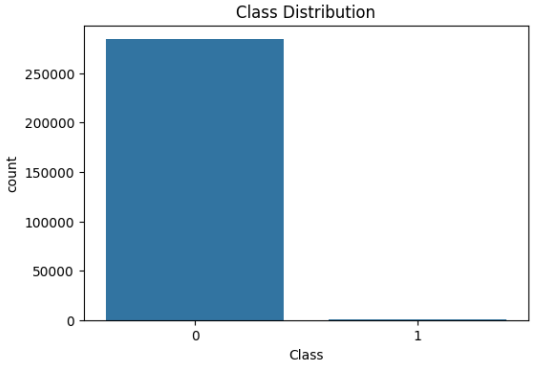


Figure 4: Class distribution of legitimate (99.83%) and fraudulent transactions (0.17%).

Because fraudulent observations represent only a very small percentage of the dataset, a model trained directly on the original data may become biased towards the majority class. In such a situation, accuracy may appear high even when fraud detection is poor. For this reason, several imbalance-handling strategies were explored rather than assuming from the start that one method would be best.

The strategies explored included:

- Random Under-Sampling,
- Random Over-Sampling,
- SMOTE,
- and class-weighted learning.

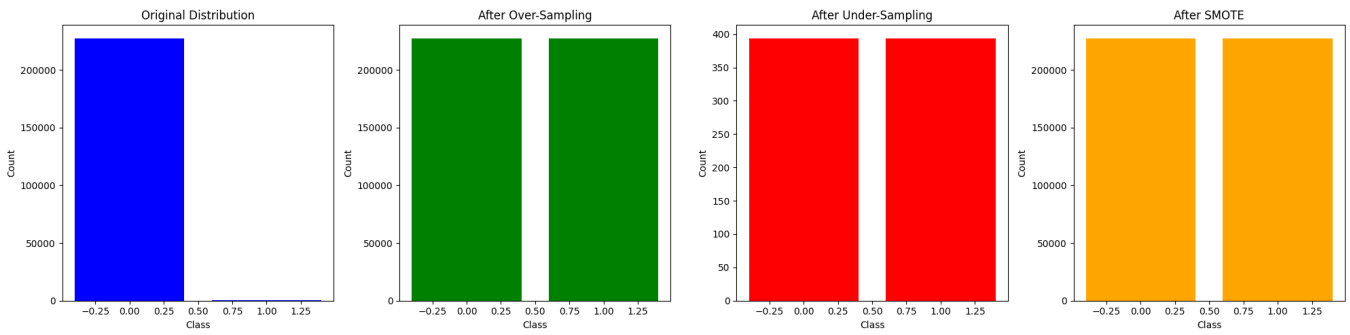


Figure 5: Training-set class distributions under the main imbalance-handling strategies explored in this project.

This design choice aligns with the broader imbalanced-learning literature, where both data-level and algorithm-level methods are commonly considered depending on the structure of the problem (Chawla et al., 2002; He

and Garcia, 2009). **All strategies were explored and retained as principal preprocessing approaches, except for Random Under-Sampling.**

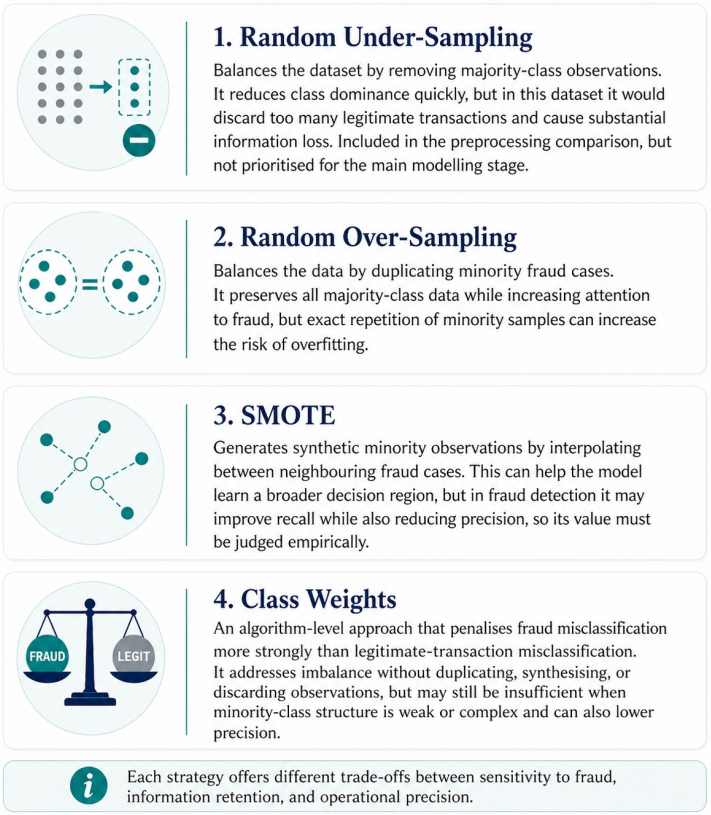


Figure 6: Imbalance-handling strategies explored during preprocessing.

The figure explains and summarises the four approaches considered for addressing the severe fraud-class imbalance: random under-sampling, random over-sampling, SMOTE, and class-weighted learning, highlighting the main trade-offs between sensitivity to fraud, information retention, overfitting risk, and operational precision.

7 Feature-Engineering Decision

No major additional engineered features were retained in the final preprocessing pipeline.

This decision was made deliberately. Most predictors in the dataset are already anonymised PCA-based variables (`V1`–`V28`), which means that they are not raw domain features in the usual sense. Adding further transformations without a clear theoretical basis risked creating features that were difficult to interpret, redundant, or insufficiently justified. The features `Time` and `Amount` were retained in their original form.

This means that the preprocessing strategy in this project prioritised **robust preparation of the existing feature space** over speculative feature creation. Given the anonymised structure of the dataset and the strong preprocessing demands already imposed by imbalance, this was considered the most defensible methodological choice.

8 Impact of Preprocessing Choices on Downstream Model Performance

The preprocessing decisions made in this section had a direct impact on later model behaviour. The absence of missing values and categorical features simplified the pipeline, while the decision not to retain IQR-based

filtering preserved anomalous transaction amounts that may contain useful fraud-related signal. The choice of `StandardScaler` also supported the scale-sensitive models explored later in the project and made comparison across model families more consistent.

Most importantly, the class-imbalance strategies changed the later precision–recall trade-off in different ways:

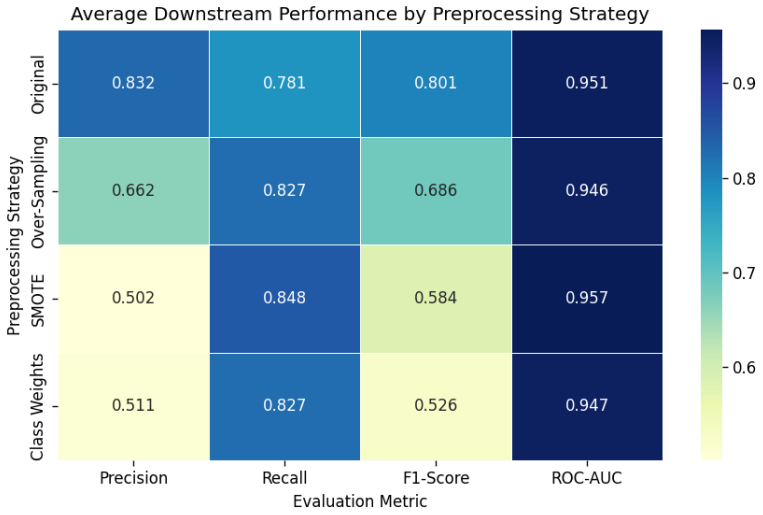


Figure 7: Average downstream model performance under the main preprocessing strategies explored in this project.

The figure aggregates the average precision, recall, F1-score, and ROC-AUC obtained across the tested classifiers in the subsequent model-development stage, showing how different imbalance-handling strategies changed the overall performance profile. Random Under-Sampling was not retained for the main modelling stage because the information loss was too severe. Random Over-Sampling, SMOTE, and class weights often increased sensitivity to fraud, but this was sometimes achieved at the cost of lower precision and weaker overall balance. Class weights provided a useful cost-sensitive alternative because they increased minority-class attention without altering the dataset itself, but they did not consistently improve the overall precision–recall balance. In later comparisons, the strongest-performing models ultimately achieved their best overall balance on the **original scaled dataset**, while the imbalance-handling strategies remained valuable as comparative alternatives rather than automatic improvements.

This is an important conclusion for this section: preprocessing does not improve performance automatically. Instead, it changes the trade-off between fraud capture and false alarms, and those trade-offs must be evaluated empirically in relation to the problem context.

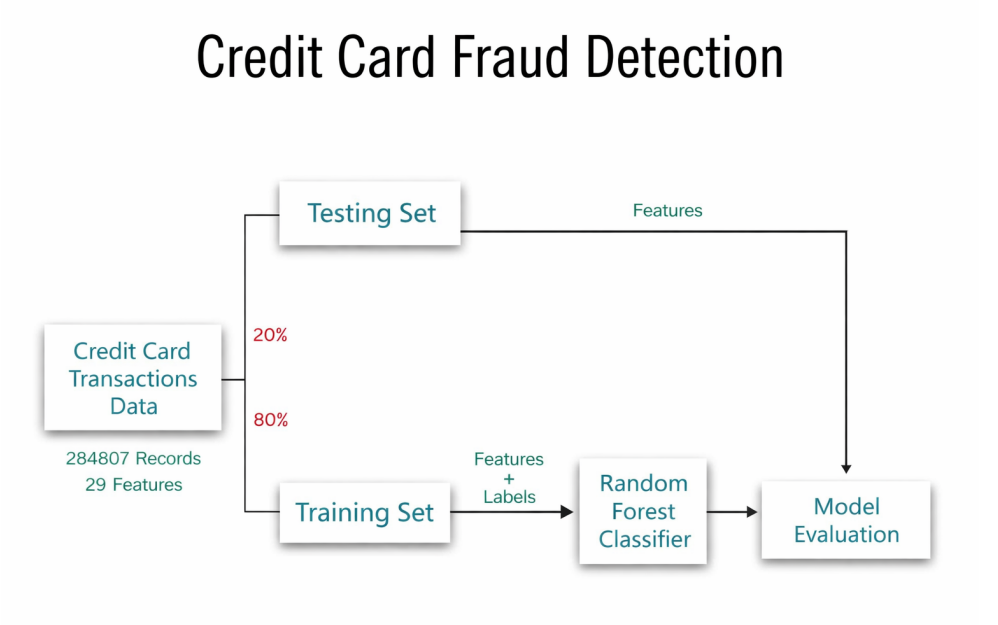
9 Conclusion

The preprocessing stage translated the main EDA findings into a justified preprocessing workflow for fraud detection. No missing-value imputation or categorical encoding was required, IQR-based outlier filtering was explored but not retained, and the data was split into features and target before a stratified train–test split. `StandardScaler` was retained as the main scaling method, several imbalance-handling strategies were explored, and no major additional feature engineering was introduced.

Taken together, these choices produced a preprocessing pipeline that was methodologically defensible, aligned with the structure of the dataset, and suitable for the later modelling stages. Most importantly, the pipeline preserved evaluation realism while allowing different imbalance-handling strategies to be compared meaningfully in a realistic fraud-detection setting.

Technical Portfolio Report

Model Development and Selection



Section Contents

1 Introduction **26**

2 Evaluation Framework **26**

3 Training Strategies Taken Forward to Model Comparison **27**

4 Candidate Models Explored **27**
4.1 Models discarded from the main final comparison 28

5 Stage 1: Broad Comparative Training **28**
5.1 First ranking stage: models selected for Cross-Validation 30

6 Stage 2: 5-Fold Cross-Validation **31**
6.1 Second ranking stage: models selected for tuning 31

7 Hyperparameter-Tuning Strategy **33**
7.1 Tuned XGBoost + Original 33
7.2 Tuned Random Forest + Original 34
7.3 Additional tuning checks: Random Forest + SMOTE and KNN + Original 35

8 Supplementary Threshold Tuning for the Finalist Tree-Based Models **35**

9 Final Model Selection and Justification **36**

10 Conclusion **37**

1 Introduction

The model-development stage focuses on selecting and training machine-learning models for the fraud-detection problem. Building on the exploratory analysis and preprocessing stage, where the dataset was analysed and a leakage-controlled preprocessing pipeline was established, this section evaluates different classifiers to identify the most suitable approach.

Rather than selecting only two models from the start, several candidate models and imbalance-handling strategies were first compared. The strongest models were then shortlisted for cross-validation and hyperparameter tuning. Because fraud cases are highly imbalanced, model selection was not based on a single metric or test split. Instead, the final choice considered predictive performance, the balance between precision and recall, robustness across validation folds, computational practicality, and suitability for a realistic fraud-detection setting.

2 Evaluation Framework

Because this is a highly imbalanced fraud-detection problem, model evaluation cannot rely on accuracy alone. A classifier may achieve very high accuracy simply by predicting the majority non-fraud class most of the time while still missing many fraud cases. For this reason, the modelling stage used a broader evaluation framework based on the **confusion matrix**, **precision**, **recall**, **F1-score**, and **ROC-AUC**. These are more informative than accuracy alone in a minority-class fraud setting. (Dal Pozzolo et al., 2018; scikit-learn developers, 2026a,c)

For binary fraud classification, the confusion matrix can be interpreted through four possible outcomes:

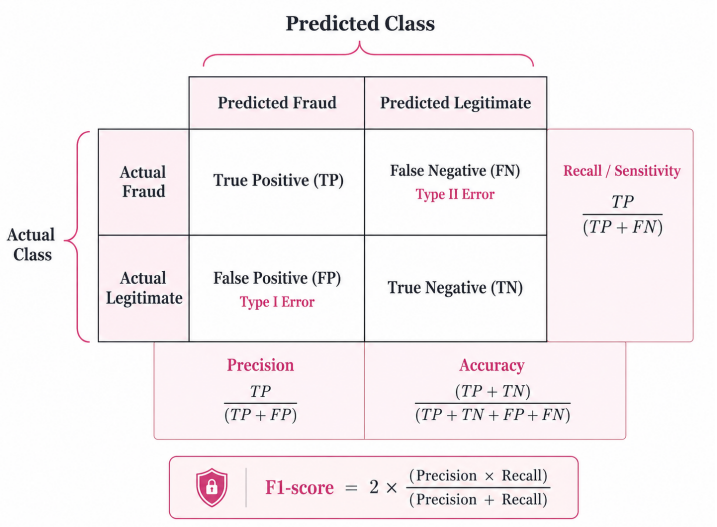


Figure 1: Confusion matrix outcomes for binary fraud classification.

Confusion matrix structure explained: **True Positive (TP)**, a fraudulent transaction correctly predicted as fraud; **True Negative (TN)**, a legitimate transaction correctly predicted as legitimate; **False Positive (FP)**, a legitimate transaction incorrectly predicted as fraud; and **False Negative (FN)**, a fraudulent transaction incorrectly predicted as legitimate. In fraud detection, **false negatives** are especially important because they represent fraud cases that escape detection. However, **false positives** also matter, because too many false alarms create operational cost and inconvenience legitimate customers. Among the scalar metrics, **precision** measures the reliability of fraud predictions, **recall** measures how many real fraud cases are captured, **F1-score** summarises the balance between the two, and **ROC-AUC** gives a threshold-independent measure of class separability. For this reason, the final comparison in this project was based on the combination of precision, recall, F1-score, ROC-AUC, and the confusion matrix rather than on accuracy alone.

3 Training Strategies Taken Forward to Model Comparison

In line with the preprocessing stage, the model-comparison stage did not assume in advance that one imbalance-handling strategy would be best. Instead, four strategies were taken forward:

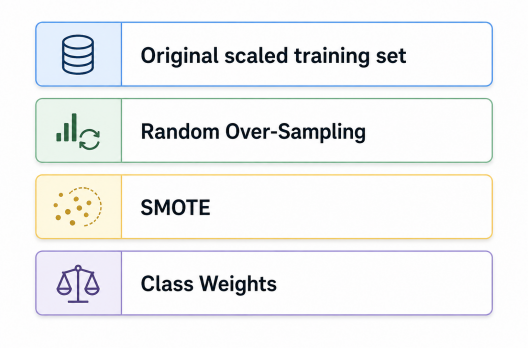


Figure 2: Imbalance-handling strategies retained.

Throughout this report, the term **Original** refers to the **original scaled training set**, that is, the leakage-safe dataset produced after stratified splitting and `StandardScaler` during the preprocessing stage.

4 Candidate Models Explored

The following classification models were explored in this stage and compared in this order:

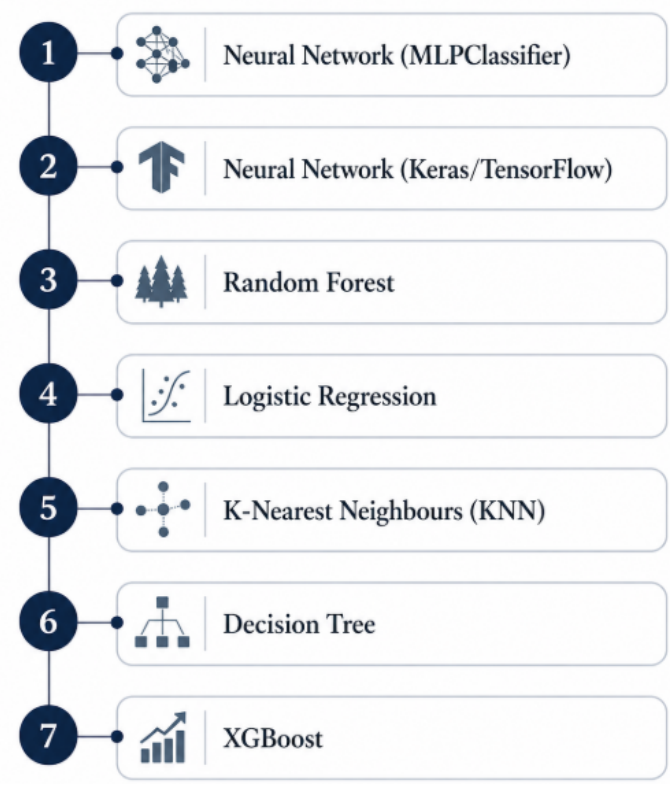


Figure 3: Classification models explored.

These models were selected because they represent different learning behaviours and together provide a meaningful basis for comparison on tabular fraud data. Logistic Regression served as an interpretable linear baseline. Decision Tree provided a simple non-linear rule-based model. Random Forest represented a strong bagging-based ensemble (Breiman, 2001). XGBoost represented a powerful gradient-boosting method (Chen and Guestrin, 2016) that is often particularly strong on tabular datasets. KNN provided a distance-based comparison model that is especially sensitive to scaling. Finally, Neural Networks (NN) were explored in two forms: a simple `MLPClassifier` baseline and a more flexible Keras implementation.

4.1 Models discarded from the main final comparison

Some additional models were also considered or partially explored, but they were not retained as part of the main final comparison for methodological and practical reasons. Support Vector Machine (SVM) was considered because it is often strong for binary classification:

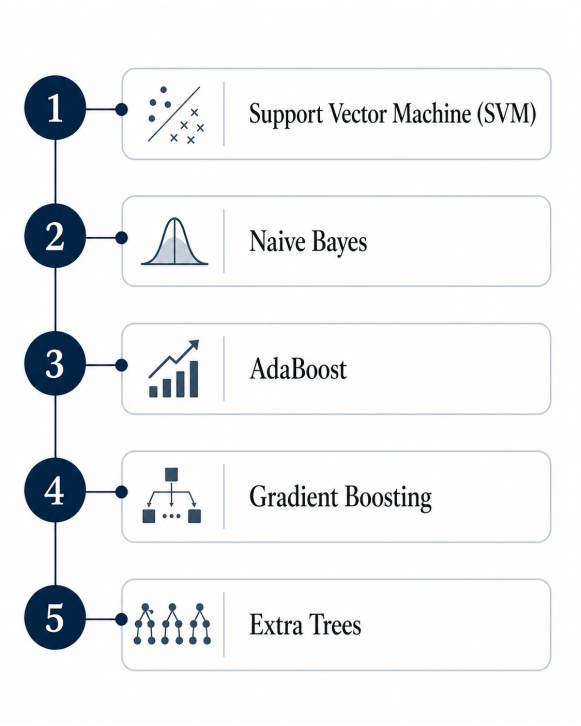


Figure 4: Models excluded from the final comparison.

However, on this dataset it was computationally too expensive to run reliably on the full data in Colab, so it was not retained in the main comparison. **Naive Bayes** was not prioritised because its strong conditional-independence assumptions were not especially well suited to this fraud-detection setting, where the signal is subtle and likely depends on relationships across multiple variables. Other ensemble methods such as **AdaBoost**, **Gradient Boosting**, and **Extra Trees** were not prioritised because the project already included a sufficiently broad and methodologically stronger comparison through Random Forest and XGBoost, while the available time for training and evaluation was limited.

5 Stage 1: Broad Comparative Training

The first stage of model selection consisted of training the candidate models across the main imbalance-handling strategies and comparing their test-split performance. This stage served as a broad filter rather than as the final decision point.

Figure 5: Stage 1 broad comparative-training workflow.

Several useful patterns emerged. First, imbalance handling was *not* automatically beneficial. Some configurations improved recall substantially but caused a major loss of precision and F1-score. This was especially clear for Logistic Regression with Over-Sampling, SMOTE, and Class Weights, where the model became much more aggressive in flagging fraud but at the cost of many more false positives. Decision Tree behaved more credibly than Logistic Regression in several cases, but its resampled versions did not consistently improve the overall balance. KNN trained on the original scaled dataset performed better than expected and was strong enough to remain interesting, but it was still not expected to outperform the strongest ensemble models. The two neural-network experiments were also informative. The simple `MLPClassifier` produced credible results, especially in recall and ROC-AUC, but its overall balance remained weaker than that of the strongest tree-based models. It also had an important practical limitation: `MLPClassifier` does not directly support class weights. This motivated a second neural-network experiment using Keras/TensorFlow, which allowed a deeper architecture, dropout, batch normalisation, early stopping, direct class weighting, and explicit threshold comparison (Figure 6).

	Threshold	Accuracy	Precision	Recall	F1-Score	TN	FP	FN	TP
0	0.5	0.999350	0.821053	0.795918	0.808290	56847	17	20	78
1	0.6	0.999403	0.855556	0.785714	0.819149	56851	13	21	77
2	0.7	0.999403	0.872093	0.765306	0.815217	56853	11	23	75
3	0.8	0.999263	0.924242	0.622449	0.743902	56859	5	37	61

Figure 6: Threshold comparison on the original Keras model.

It shows that **0.6** produced the most balanced result, improving precision and F1-score relative to the default threshold of 0.5 while preserving acceptable recall. Among the Keras variants, the **original scaled training set** provided the best overall balance. The Keras model was trained for up to `epochs=30` with early stopping based on validation loss (`patience = 5`). This means that training could stop before the maximum number of epochs if validation performance stopped improving, while `restore_best_weights=True` ensured that the final model retained the weights from the best epoch rather than from the last epoch. Even so, although the Keras model was credible, it still did not surpass the strongest tree-based models overall.

A small practical note also emerged for Logistic Regression: increasing `max_iter` beyond the standard setting removed optimisation warnings but did not materially improve the evaluation metrics. This confirmed that the model's limitations here were not mainly due to insufficient iterations, but to its weaker fit to the structure of the problem.

Key Insight

Although NN can learn complex patterns automatically and may require less manual feature engineering in some domains, they do not remove the need for preprocessing. In this project, standardisation remained important for stable optimisation on tabular data, while class imbalance handling and threshold selection also had a direct effect on the balance between fraud detection and false positives.

5.1 First ranking stage: models selected for Cross-Validation

After the broad comparison, the project applied a first filtering and ranking stage to identify the strongest model–strategy combinations to take forward to the next stage. The main filter used minimum thresholds on **precision**, **recall**, **F1-score**, and **ROC-AUC**, together with a weighted business-oriented score that slightly prioritised recall while still rewarding precision and overall balance.

	Model	Training Strategy	Accuracy	Precision	Recall	F1-Score	ROC-AUC	TN	FP	FN	TP	Pass_Filter	Business_Score
0	Random Forest	Original	0.999596	0.941176	0.816327	0.874317	0.963027	56859.0	5.0	18.0	80.0	True	0.882949
1	XGBoost	Original	0.999544	0.891304	0.836735	0.863158	0.975397	56854.0	10.0	16.0	82.0	True	0.873578
2	KNN	Original	0.999544	0.918605	0.806122	0.858696	0.943743	56857.0	7.0	19.0	79.0	True	0.866772
3	Random Forest	SMOTE	0.999491	0.870968	0.826531	0.848168	0.968448	56852.0	12.0	17.0	81.0	True	0.859463
4	Random Forest	Over-Sampling	0.999526	0.949367	0.765306	0.847458	0.962822	56860.0	4.0	23.0	75.0	False	0.860814
5	Random Forest	Class Weights	0.999508	0.960526	0.744898	0.839080	0.952905	56861.0	3.0	25.0	73.0	False	0.853933
6	Neural Network (Keras)	Original	0.999403	0.855556	0.785714	0.819149	0.964899	56851.0	13.0	21.0	77.0	False	0.832944
7	XGBoost	Over-Sampling	0.999315	0.770642	0.857143	0.811594	0.972226	56839.0	25.0	14.0	84.0	False	0.831314
8	XGBoost	Class Weights	0.999280	0.761468	0.846939	0.801932	0.974753	56838.0	26.0	15.0	83.0	False	0.822827
9	Neural Network (Keras)	Over-Sampling	0.999228	0.745455	0.836735	0.788462	0.960739	56836.0	28.0	16.0	82.0	False	0.809683
10	Neural Network (MLPClassifier)	Over-Sampling	0.999192	0.732143	0.836735	0.780952	0.961519	56834.0	30.0	16.0	82.0	False	0.803890
11	Neural Network (MLPClassifier)	SMOTE	0.999228	0.764706	0.795918	0.780000	0.965368	56840.0	24.0	20.0	78.0	False	0.799520
12	KNN	Over-Sampling	0.999017	0.664062	0.867347	0.752212	0.943667	56821.0	43.0	13.0	85.0	False	0.785210
13	Neural Network (MLPClassifier)	Original	0.998912	0.638462	0.846939	0.728070	0.975034	56817.0	47.0	15.0	83.0	False	0.767488
14	Neural Network (Keras)	SMOTE	0.998824	0.611511	0.867347	0.717300	0.973635	56810.0	54.0	13.0	85.0	False	0.763713
15	Decision Tree	Original	0.999140	0.752577	0.744898	0.748718	0.872238	56840.0	24.0	25.0	73.0	False	0.760891
16	Logistic Regression	Original	0.999140	0.826667	0.632653	0.716763	0.960549	56851.0	13.0	36.0	62.0	False	0.744674
17	Decision Tree	Over-Sampling	0.998999	0.711340	0.704082	0.707692	0.851795	56836.0	28.0	29.0	69.0	False	0.721933
18	Decision Tree	Class Weights	0.998929	0.676190	0.724490	0.699507	0.861946	56830.0	34.0	27.0	71.0	False	0.717500
19	KNN	SMOTE	0.997841	0.436548	0.877551	0.583051	0.948415	56753.0	111.0	12.0	86.0	False	0.678712
20	XGBoost	SMOTE	0.997296	0.376106	0.867347	0.524691	0.977248	56723.0	141.0	13.0	85.0	False	0.645301
21	Decision Tree	SMOTE	0.997560	0.394872	0.785714	0.525597	0.891820	56746.0	118.0	21.0	77.0	False	0.614043
22	Neural Network (Keras)	Class Weights	0.985394	0.096703	0.897959	0.174603	0.972960	56042.0	822.0	10.0	88.0	False	0.484243
23	Logistic Regression	Over-Sampling	0.975668	0.061308	0.918367	0.114943	0.972004	55486.0	1378.0	8.0	90.0	False	0.465757
24	Logistic Regression	Class Weights	0.975528	0.060976	0.918367	0.114358	0.972083	55478.0	1386.0	8.0	90.0	False	0.465519
25	Logistic Regression	SMOTE	0.974106	0.057803	0.918367	0.108761	0.970843	55397.0	1467.0	8.0	90.0	False	0.463044

(a) Full ranking of model–strategy combinations.

```
=== Shortlisted combinations (passed the filter) ===
```

	Model	Training Strategy	Accuracy	Precision	Recall	F1-Score	ROC-AUC	TN	FP	FN	TP	Pass_Filter	Business_Score
0	Random Forest	Original	0.999596	0.941176	0.816327	0.874317	0.963027	56859.0	5.0	18.0	80.0	True	0.882949
1	XGBoost	Original	0.999544	0.891304	0.836735	0.863158	0.975397	56854.0	10.0	16.0	82.0	True	0.873578
2	KNN	Original	0.999544	0.918605	0.806122	0.858696	0.943743	56857.0	7.0	19.0	79.0	True	0.866772
3	Random Forest	SMOTE	0.999491	0.870968	0.826531	0.848168	0.968448	56852.0	12.0	17.0	81.0	True	0.859463

```
=== Best combination for each model ===
```

	Model	Training Strategy	Accuracy	Precision	Recall	F1-Score	ROC-AUC	TN	FP	FN	TP	Pass_Filter	Business_Score
0	Decision Tree	Original	0.999140	0.752577	0.744898	0.748718	0.872238	56840.0	24.0	25.0	73.0	False	0.760891
1	KNN	Original	0.999544	0.918605	0.806122	0.858696	0.943743	56857.0	7.0	19.0	79.0	True	0.866772
2	Logistic Regression	Original	0.999140	0.826667	0.632653	0.716763	0.960549	56851.0	13.0	36.0	62.0	False	0.744674
3	Neural Network (Keras)	Original	0.999403	0.855556	0.785714	0.819149	0.964899	56851.0	13.0	21.0	77.0	False	0.832944
4	Neural Network (MLPClassifier)	Over-Sampling	0.999192	0.732143	0.836735	0.780952	0.961519	56834.0	30.0	16.0	82.0	False	0.803890
5	Random Forest	Original	0.999596	0.941176	0.816327	0.874317	0.963027	56859.0	5.0	18.0	80.0	True	0.882949
6	XGBoost	Original	0.999544	0.891304	0.836735	0.863158	0.975397	56854.0	10.0	16.0	82.0	True	0.873578

```
=== Top 5 overall combinations ===
```

	Model	Training Strategy	Accuracy	Precision	Recall	F1-Score	ROC-AUC	TN	FP	FN	TP	Pass_Filter	Business_Score
0	Random Forest	Original	0.999596	0.941176	0.816327	0.874317	0.963027	56859.0	5.0	18.0	80.0	True	0.882949
1	XGBoost	Original	0.999544	0.891304	0.836735	0.863158	0.975397	56854.0	10.0	16.0	82.0	True	0.873578
2	KNN	Original	0.999544	0.918605	0.806122	0.858696	0.943743	56857.0	7.0	19.0	79.0	True	0.866772
3	Random Forest	SMOTE	0.999491	0.870968	0.826531	0.848168	0.968448	56852.0	12.0	17.0	81.0	True	0.859463
4	Random Forest	Over-Sampling	0.999526	0.949367	0.765306	0.847458	0.962822	56860.0	4.0	23.0	75.0	False	0.860814

(b) Shortlisted and leading model–strategy combinations.

Figure 7: Stage 1 model ranking from the initial test split.

Subfigure (a) reports the full ranking of all model–training strategy combinations, ordered using the pass/fail shortlisting filter and the weighted business score. Subfigure (b) summarises the shortlisted combinations that passed the filter, the best-performing configuration for each model, and the top five overall configurations. The shortlisting filter required Precision, Recall, and F1-Score to be at least 0.80, and ROC-AUC to be at least 0.94. The business score prioritised Recall while still accounting for Precision, F1-Score, and ROC-AUC, using weights of 0.35, 0.30, 0.25, and 0.10 respectively.

$$Business_Score = 0.35 \cdot Recall + 0.30 \cdot Precision + 0.25 \cdot F1\text{-}Score + 0.10 \cdot ROC\text{-}AUC \tag{5.1}$$

The main shortlisted configurations from this stage were:

- **Random Forest + Original**
- **XGBoost + Original**
- **KNN + Original**
- **Random Forest + SMOTE**

In addition, the strongest Keras configuration (**Keras + Original with threshold 0.6**) was retained as an extra confirmation experiment, even though it did not rank as one of the top principal finalists.

6 Stage 2: 5-Fold Cross-Validation

After the first ranking stage, the strongest candidates were taken forward for formal cross-validation using **Stratified 5-Fold**. Stratification was necessary because the fraud class is extremely rare, and the class proportions therefore needed to remain similar across folds.

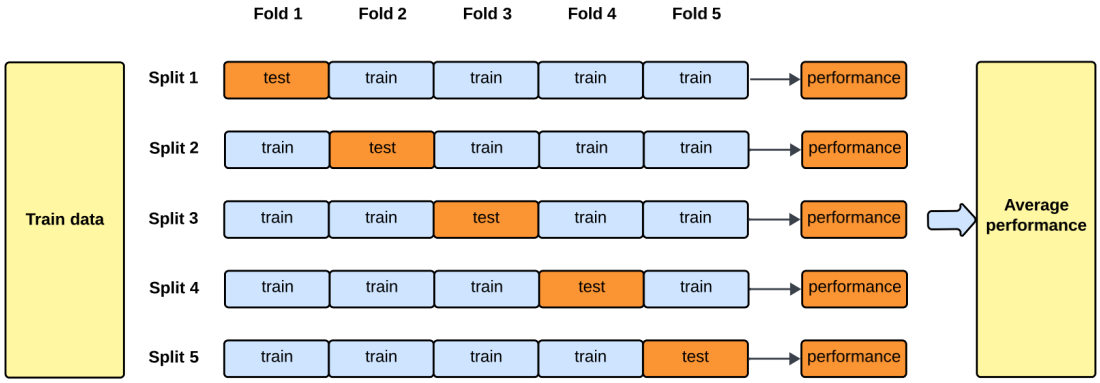


Figure 8: Five-fold cross-validation workflow.

Cross-validation served two purposes. First, it reduced dependence on one particular train–test split. Second, it provided a more robust estimate of performance stability by showing how the shortlisted models behaved across multiple partitions of the training data.

A special methodological issue arose for **Random Forest + SMOTE**. SMOTE could not be applied once to the full training set before cross-validation, because that would create data leakage by allowing synthetic samples to incorporate information from what should later act as validation folds. For this reason, the SMOTE version was cross-validated only through a pipeline in which SMOTE was applied separately inside each fold (scikit-learn developers, 2026d).

6.1 Second ranking stage: models selected for tuning

After cross-validation, a second filtering and ranking step was applied to decide which models should proceed to formal hyperparameter tuning. This stage used minimum thresholds for Precision, Recall, F1-Score, and ROC-AUC, together with the same business-oriented weighted score used in the initial ranking, but adjusted with a small penalty for instability across folds.

```
=== FILTERS USED ===
Precision >= 0.88
Recall    >= 0.77
F1-Score  >= 0.8
ROC-AUC   >= 0.94

=== ALL K-FOLD MODELS RANKED ===
```

	Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC	Std_Prec	Std_Rec	Std_F1	Pass_Filter	Final_Score
0	XGBoost + Original	0.999548	0.939975	0.791918	0.858261	0.977005	0.040765	0.033342	0.012548	True	0.862764
1	Random Forest + Original	0.999530	0.946530	0.774067	0.850747	0.948841	0.035098	0.024954	0.008723	True	0.855576
2	Random Forest + SMOTE	0.999451	0.880987	0.791886	0.832732	0.978415	0.030975	0.041239	0.017432	True	0.838516
3	KNN + Original	0.999478	0.932338	0.753781	0.833007	0.919893	0.030281	0.023722	0.014149	False	0.836950
4	Keras NN + Original (thr=0.6)	0.999372	0.875904	0.743687	0.802228	0.971311	0.023627	0.068463	0.038276	False	0.807713

```
=== SHORTLISTED MODELS (PASSED FILTER) ===
```

	Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC	Std_Prec	Std_Rec	Std_F1	Final_Score
0	XGBoost + Original	0.999548	0.939975	0.791918	0.858261	0.977005	0.040765	0.033342	0.012548	0.862764
1	Random Forest + Original	0.999530	0.946530	0.774067	0.850747	0.948841	0.035098	0.024954	0.008723	0.855576
2	Random Forest + SMOTE	0.999451	0.880987	0.791886	0.832732	0.978415	0.030975	0.041239	0.017432	0.838516

```
=== RECOMMENDED MODELS TO TUNE ===
```

	Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC	Std_Prec	Std_Rec	Std_F1	Final_Score
0	XGBoost + Original	0.999548	0.939975	0.791918	0.858261	0.977005	0.040765	0.033342	0.012548	0.862764
1	Random Forest + Original	0.999530	0.946530	0.774067	0.850747	0.948841	0.035098	0.024954	0.008723	0.855576

Figure 9: Cross-validation summary for the shortlisted models.

After ranking all model-strategy combinations with a business-oriented score prioritising Recall, the strongest candidates were evaluated using 5-fold cross-validation. Since no model reached the original mean Recall threshold of 0.80, this threshold was relaxed to 0.77 only for candidate selection before hyperparameter tuning. Strict thresholds for Precision, F1-Score and ROC-AUC were maintained, and a Final Score was used to penalise instability across folds. Based on this balance between performance and stability, XGBoost Original and Random Forest Original were selected for tuning.

$$Final_Score = Business_Score - Stability_Penalty \tag{6.1}$$

where:

$$Stability_Penalty = 0.10 \cdot (Std_{Precision} + Std_{Recall} + Std_{F1}) \tag{6.2}$$

The cross-validation results confirmed **Random Forest + Original** and **XGBoost + Original** as the strongest candidates, offering the best balance between performance and stability. **Random Forest + SMOTE** and **KNN + Original** were kept as useful comparison points, but not selected for main tuning. The Keras model remained credible, although its Recall, F1-Score, and fold stability were weaker than those of the leading tree-based models.

Based on this second-stage ranking, the models selected for formal hyperparameter tuning were:

- **Random Forest + Original**
- **XGBoost + Original**

Key Insight

Although Random Forest is often considered relatively robust and can sometimes be assessed with internal tools such as out-of-bag error, this does not make cross-validation unnecessary. In this project, 5-fold stratified cross-validation was still applied in order to obtain a more reliable estimate of performance and to keep the evaluation consistent with the other shortlisted models.

7 Hyperparameter-Tuning Strategy

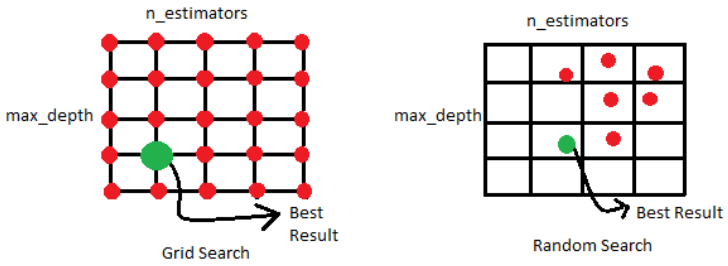


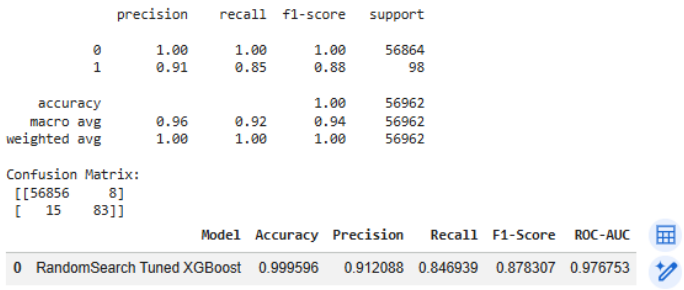
Figure 10: Comparison between grid search and random search for hyperparameter tuning.

Grid search evaluates all predefined combinations of parameters, while random search samples only selected combinations from the search space, often reducing computational cost while still identifying strong-performing configurations. Once the final candidates had been validated under 5-fold cross-validation, hyperparameter tuning was applied. In this project, **RandomizedSearchCV** was selected as the main tuning strategy. (scikit-learn developers, 2026g,b) This choice was justified because the search spaces for ensemble models become large quickly and the project was run in a notebook-based Colab environment with finite computational resources. The main tuning objective was **F1-score**. This was a deliberate decision. Fraud detection requires a balance between precision and recall, so tuning for F1-score was more appropriate than tuning for accuracy alone.

7.1 Tuned XGBoost + Original

Hyperparameter tuning was first applied to **XGBoost trained on the original scaled dataset**. A **RandomizedSearchCV** procedure with 5-fold stratified cross-validation was used over a search space including:

- `n_estimators`
- `max_depth`
- `learning_rate`
- `subsample`
- `colsample_bytree`
- `min_child_weight`



	precision	recall	f1-score	support
0	1.00	1.00	1.00	56864
1	0.91	0.85	0.88	98
accuracy			1.00	56962
macro avg	0.96	0.92	0.94	56962
weighted avg	1.00	1.00	1.00	56962

Confusion Matrix:
[[56856 8]
[15 83]]

	Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC
0	RandomSearch Tuned XGBoost	0.999596	0.912088	0.846939	0.878307	0.976753

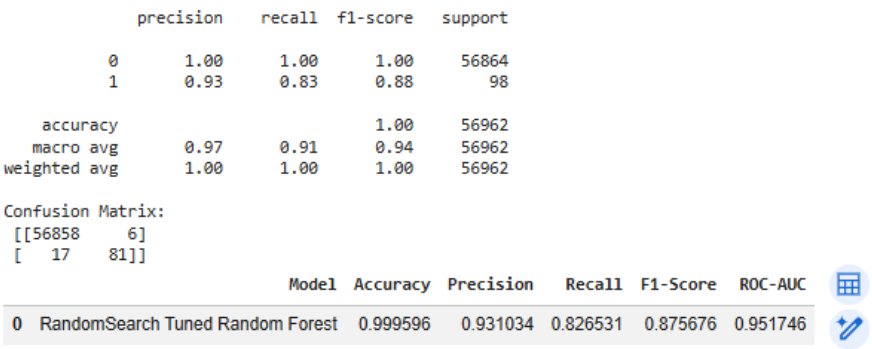
Figure 11: Tuned XGBoost on the original scaled data.

This tuning stage produced a clearly strong final XGBoost model and remained computationally efficient in Colab. A later, more exhaustive **GridSearchCV** was also tested on the same original scaled training set in order to check whether a larger search would produce a meaningful improvement. However, the gain over the earlier random search was negligible in practical terms, while the computational cost was much higher. This showed that the earlier RandomizedSearchCV had already identified a near-optimal region of the parameter space and that the larger grid search was not proportionate to the additional computational burden. This was an important result because it showed that XGBoost was not only strong at baseline, but also responsive to tuning in a computationally efficient way.

7.2 Tuned Random Forest + Original

Hyperparameter tuning was also applied to **Random Forest trained on the original scaled dataset**. Because Random Forest was noticeably more expensive to train repeatedly than XGBoost in this notebook environment, a smaller **RandomizedSearchCV** search space was used. The main tuning parameters included:

- `n_estimators`
- `max_depth`
- `min_samples_split`
- `min_samples_leaf`
- `max_features`



	precision	recall	f1-score	support
0	1.00	1.00	1.00	56864
1	0.93	0.83	0.88	98
accuracy			1.00	56962
macro avg	0.97	0.91	0.94	56962
weighted avg	1.00	1.00	1.00	56962

Confusion Matrix:
[[56858 6]
[17 81]]

	Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC
0	RandomSearch Tuned Random Forest	0.999596	0.931034	0.826531	0.875676	0.951746

Figure 12: Tuned Random Forest on the original scaled data.

A methodological detail also arose here. The initial search focused on `max_features = "sqrt"` in order to keep the process computationally feasible. Later, `"log2"` was also tested to see whether a broader search would improve performance. Although the `log2` setting slightly improved precision and ROC-AUC, it also reduced recall and slightly lowered F1-score, making the model more conservative. Because the fraud-detection scenario required a strong balance between identifying fraud and avoiding excessive false alarms, this trade-off was not considered preferable, and the final Random Forest configuration retained the better-balanced `"sqrt"` setting.

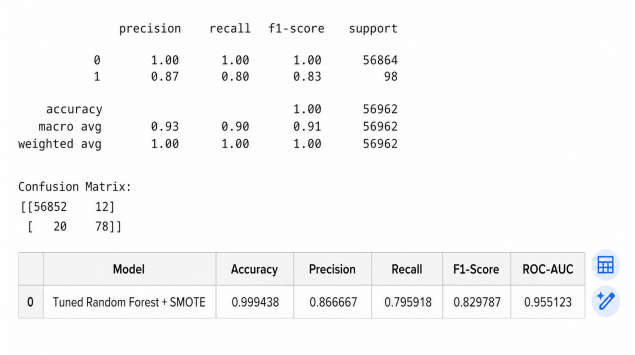
Overall, the tuned Random Forest produced only a small practical improvement over the already strong baseline version. This does not weaken Random Forest as a candidate. On the contrary, it suggests that the baseline model was already robust and well matched to the structure of the dataset.

7.3 Additional tuning checks: Random Forest + SMOTE and KNN + Original

Two additional tuning experiments were also carried out:

- **Random Forest + SMOTE**
- **KNN + Original**

These were not included because they were expected to outperform the main finalists from the start. Rather, they served as final methodological checks to confirm whether secondary candidates could improve enough under tuning to challenge the strongest models.

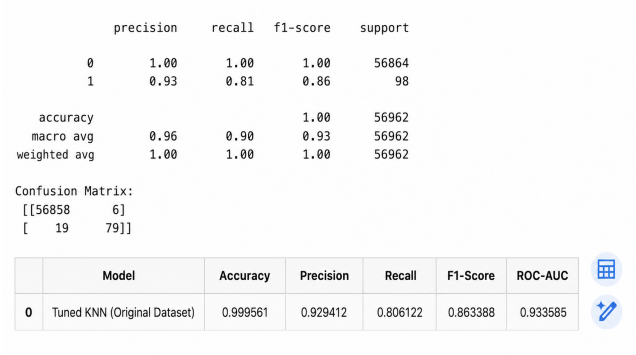


	precision	recall	f1-score	support
0	1.00	1.00	1.00	56864
1	0.87	0.80	0.83	98
accuracy			1.00	56962
macro avg	0.93	0.90	0.91	56962
weighted avg	1.00	1.00	1.00	56962

Confusion Matrix:
[[56852 12]
[20 78]]

	Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC
0	Tuned Random Forest + SMOTE	0.999438	0.866667	0.795918	0.829787	0.955123

(a) Tuned Random Forest with SMOTE.



	precision	recall	f1-score	support
0	1.00	1.00	1.00	56864
1	0.93	0.81	0.86	98
accuracy			1.00	56962
macro avg	0.96	0.90	0.93	56962
weighted avg	1.00	1.00	1.00	56962

Confusion Matrix:
[[56858 6]
[19 79]]

	Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC
0	Tuned KNN (Original Dataset)	0.999561	0.929412	0.806122	0.863388	0.933585

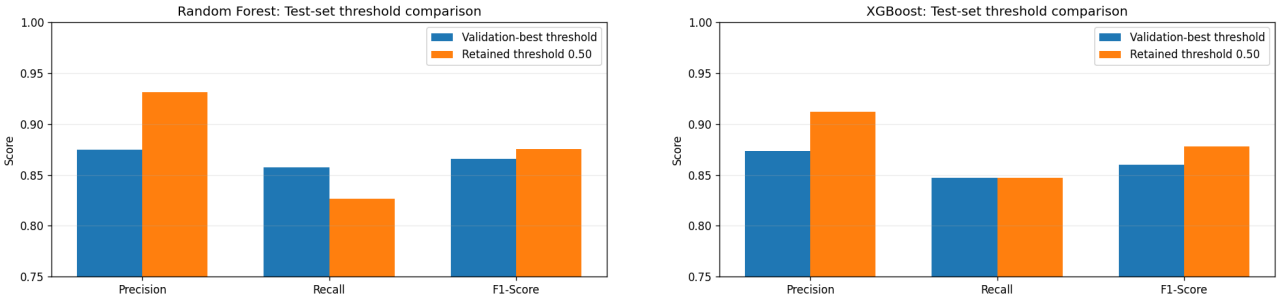
(b) Tuned KNN on the original scaled data.

Figure 13: Additional tuned model configurations.

The tuned **Random Forest + SMOTE** remained weaker than the tuned original Random Forest. In practical terms, it produced lower precision, lower recall, lower F1-score, and more false positives and false negatives than the strongest final candidates. The tuning exercise was still useful because it confirmed that the additional computational cost and resampling complexity were not justified by the resulting performance. The tuned **KNN + Original** also remained a credible comparison model, but it did not improve enough to challenge the tuned Random Forest and tuned XGBoost finalists. Its practical improvements were small relative to the computational cost, and its overall balance remained weaker than that of the best ensemble models.

8 Supplementary Threshold Tuning for the Finalist Tree-Based Models

After the tuning stage, an additional threshold-tuning experiment was carried out for **Random Forest** and **XGBoost**. The purpose was to test whether a threshold lower than the default value of 0.5 could improve the balance between fraud detection and false positives.



(a) Random Forest threshold comparison. **(b)** XGBoost threshold comparison.

Figure 14: Default and validation-best thresholds for the tuned finalists.

For Random Forest, the lower validation-best threshold slightly increased recall, but this came at the cost of lower precision and no improvement in overall balance. For XGBoost, the lower threshold did not produce a meaningful improvement in recall and instead reduced precision and F1-score. The default threshold of 0.5 was therefore retained for both models as the more stable and more defensible final decision rule. This was not treated as part of the main hyperparameter search. Instead, it was a supplementary post-training analysis. A validation split was taken from the original training data, several thresholds were tested, and the validation-optimal thresholds were then applied once to the untouched test set. This was methodologically preferable to selecting the threshold directly on the test set.

9 Final Model Selection and Justification

After broad comparison, cross-validation, hyperparameter tuning, and supplementary threshold checking, the final conclusion of the model-development stage was that the two strongest models were Random Forest and XGBoost, both trained on the original scaled dataset. Logistic Regression, Decision Tree, KNN, and the neural-network variants remained useful baselines but were not retained for final evaluation:

Figure 15: Finalist model configurations.

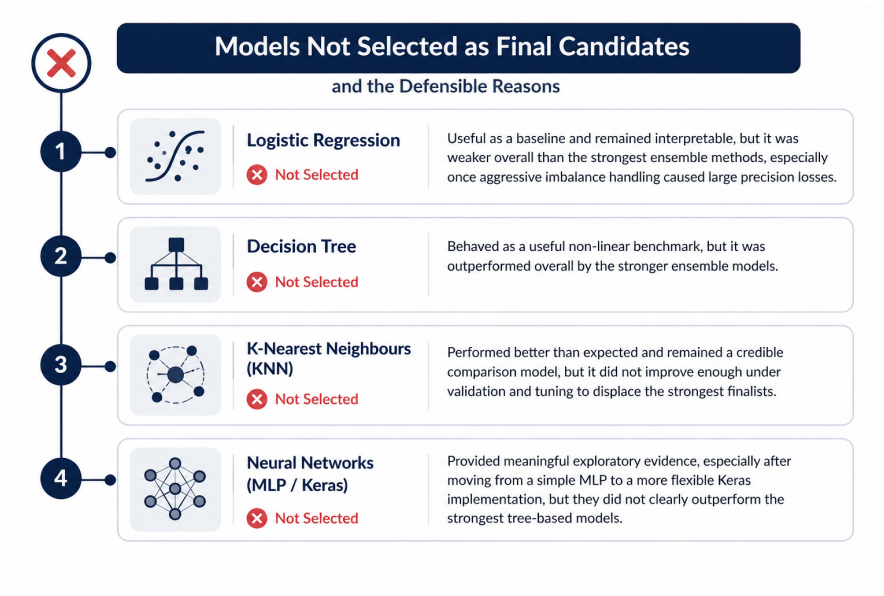


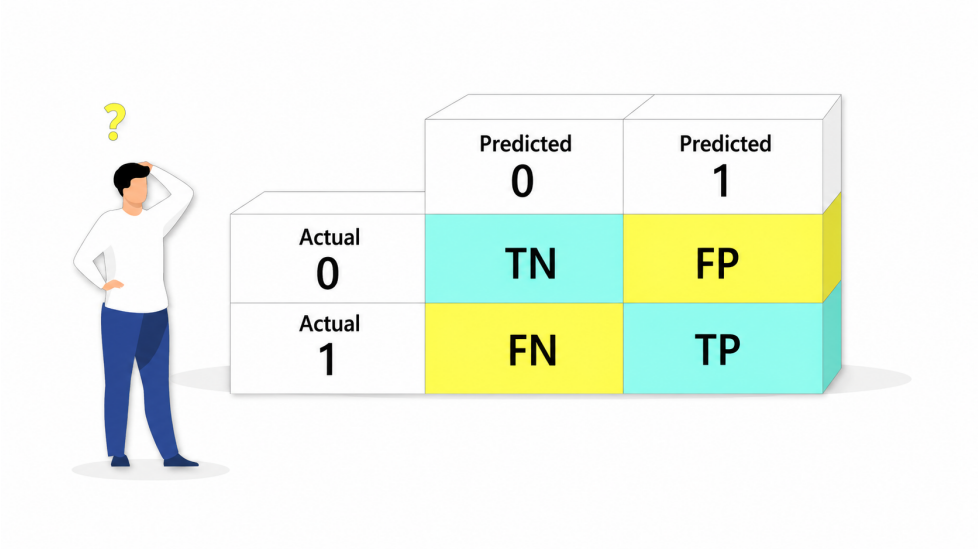
Figure 16: Models not retained for final evaluation.

10 Conclusion

The model-development stage used a staged model-selection process in which several classifiers and imbalance-handling strategies were compared before the strongest configurations were validated through 5-fold stratified cross-validation and hyperparameter tuning. The results showed that **Random Forest + Original** and **XGBoost + Original** were the most suitable models, offering the best balance between performance, stability, and practical usefulness. Other models, such as Logistic Regression, KNN, Decision Tree, and NN, provided useful baselines and comparison points. Overall, the model-development stage produced a defensible shortlist of trained and tuned models for the final evaluation stage, highlighting the importance of evidence-based model selection in fraud detection.

Technical Portfolio Report

Model Evaluation and Visualisation



Section Contents

1 Introduction **40**

2 Evaluation Logic and Business Criteria **40**

3 Final Metric Comparison of the Shortlisted Models **41**

4 Confusion-Matrix Analysis **42**

5 Precision–Recall Analysis **42**

6 ROC-Curve Analysis **43**

7 Feature Importance and Interpretability **44**
7.1 Illustrative tree visualisation 44

8 Final Model Selection Based on Business Requirements **45**

9 Conclusion **46**

1 Introduction

This section focuses on the **final evaluation, visualisation, and interpretation** of the shortlisted fraud-detection models. The model-development stage established that the two strongest candidates were **Random Forest trained on the original scaled dataset** and **XGBoost trained on the original scaled dataset**, both tuned using RandomizedSearchCV. The purpose of the present section is therefore to evaluate these finalists in a more decision-oriented way, using both scalar metrics and visual evidence, and to determine which model should be selected as the final business recommendation.

A useful final model must provide a credible balance between:

Figure 1: Final model-selection trade-offs.

This stage is especially important because, in an imbalanced fraud-detection setting, the strongest model is not necessarily the one with the highest value on a single metric. For this reason, This section evaluates the finalists using: **Precision, Recall, F1-score, and ROC-AUC, Precision–Recall behaviour, confusion matrices, ROC curves, feature-importance analysis** and a brief **interpretability discussion**

2 Evaluation Logic and Business Criteria

Given the high class imbalance in the dataset, the final evaluation cannot rely on accuracy alone. A model may achieve extremely high accuracy while still performing poorly on the minority fraud class. For this reason, the main comparison focuses on **precision**, **recall**, **F1-score**, **ROC-AUC**, and the **confusion matrix**, rather than on accuracy alone. (Dal Pozzolo et al., 2018)

As discussed in earlier analytical and modelling stages, in this fraud-detection context:

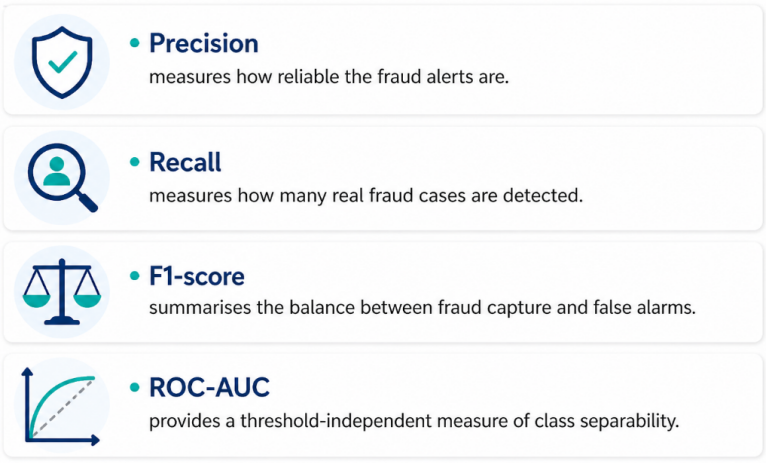


Figure 2: Fraud-detection evaluation metrics.

However, even these scalar metrics do not tell the whole story. The confusion matrix makes the trade-off operationally visible, while the Precision–Recall curve is particularly useful in strongly imbalanced problems, where it is often more informative than ROC alone (Saito and Rehmsmeier, 2015). This means that the final model will not be selected simply because it leads on one measure, but it will be selected because it provides the strongest overall balance between: sensitivity to fraud, control of false positives, threshold-independent discrimination, and practical business usefulness.

3 Final Metric Comparison of the Shortlisted Models

The final metric comparison confirmed that both **Random Forest** and **XGBoost** are highly effective classifiers for this fraud-detection problem. Both models achieved extremely high overall accuracy, which is expected in such an imbalanced dataset. However, the more informative metrics showed meaningful differences between them:

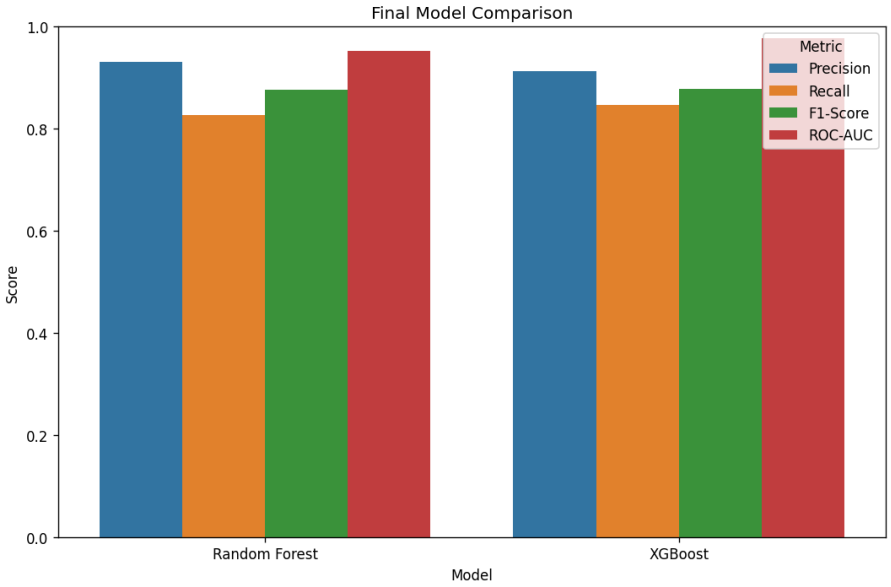


Figure 3: Final metric comparison between the two shortlisted models.

Random Forest achieved slightly stronger **precision**, which means that when it predicted fraud, it was marginally more reliable and produced slightly fewer false positives. XGBoost, on the other hand, achieved

slightly stronger **recall**, **F1-score**, **ROC-AUC**, and later also **Average Precision** in the Precision–Recall analysis. This suggests that XGBoost provided a somewhat stronger overall balance between fraud capture and classification quality, while Random Forest remained especially attractive when greater alert precision was preferred.

The main conclusion from the metric table is therefore not that one model is weak and the other is strong. Both finalists are strong, but they emphasise the fraud-detection trade-off in slightly different ways:

- **Random Forest**: slightly better precision and slightly fewer false alarms,
- **XGBoost**: slightly better fraud capture and stronger overall discrimination.

4 Confusion-Matrix Analysis

The confusion matrices provide the clearest operational view of how the two finalist models behaved on the test set. In both cases, the number of correctly classified legitimate transactions was extremely high, and the number of false positives remained very small. This confirms that both models are operationally credible and would not generate an excessive volume of unnecessary fraud alerts.

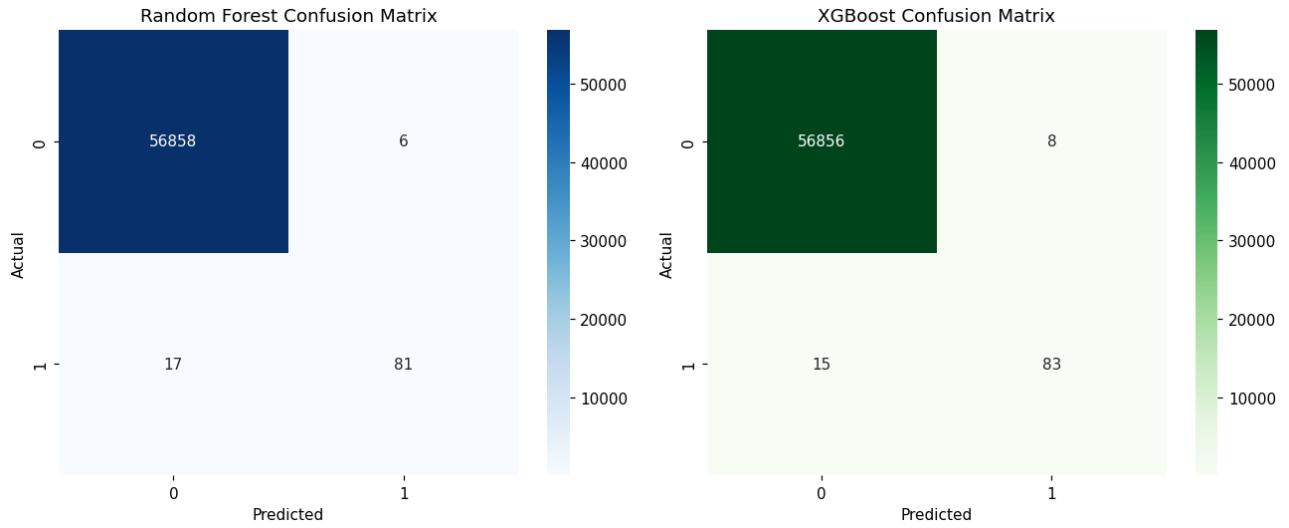


Figure 4: Confusion matrices of the finalist models.

At the same time, the confusion matrices also show that XGBoost correctly identified slightly more fraudulent transactions, while Random Forest produced slightly fewer false positives. This is consistent with the scalar metrics discussed earlier: Random Forest remained marginally stronger in precision, whereas XGBoost achieved marginally stronger recall. This is an important business trade-off. If the institution places more weight on avoiding customer inconvenience and unnecessary manual reviews, Random Forest remains attractive. If the institution places slightly more weight on capturing as many fraud cases as possible, XGBoost becomes more attractive. Since financial fraud detection usually treats missed fraud as especially costly, the stronger fraud capture of XGBoost is a meaningful advantage

5 Precision–Recall Analysis

Because the fraud class is extremely rare, the Precision–Recall analysis is especially relevant (Saito and Rehmsmeier, 2015). XGBoost achieved the stronger Average Precision, supporting its advantage in minority-

class performance.

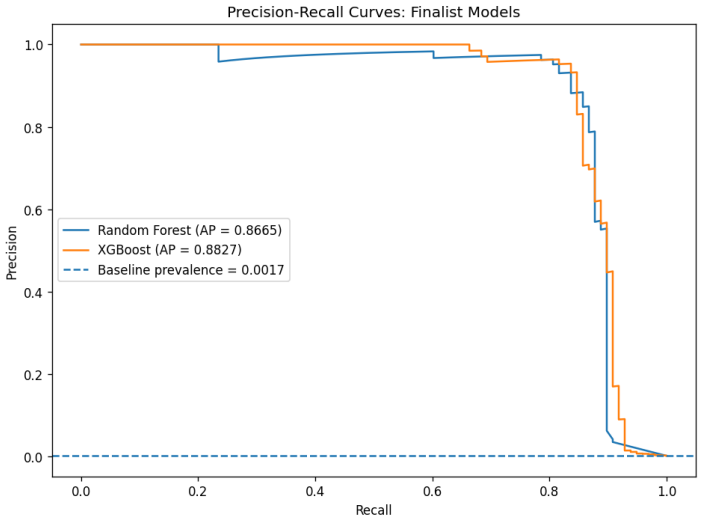


Figure 5: Precision–Recall curves for the finalist models.

The Precision–Recall curves showed that both finalist models remained strong across thresholds, but XGBoost achieved the better overall result. In particular, XGBoost obtained a higher **Average Precision** than Random Forest, indicating a stronger overall balance between fraud capture and alert reliability across decision thresholds. Random Forest remained highly competitive, but XGBoost held a small overall advantage in this more fraud-sensitive evaluation view. This is important because it strengthens the case that the superiority of XGBoost is not limited to one metric or one threshold. Instead, it appears consistently across several views of performance.

6 ROC-Curve Analysis

The ROC curves provide a threshold-independent comparison. Both models performed far above the random baseline, but XGBoost achieved the higher ROC-AUC, adding further evidence of stronger overall discrimination:

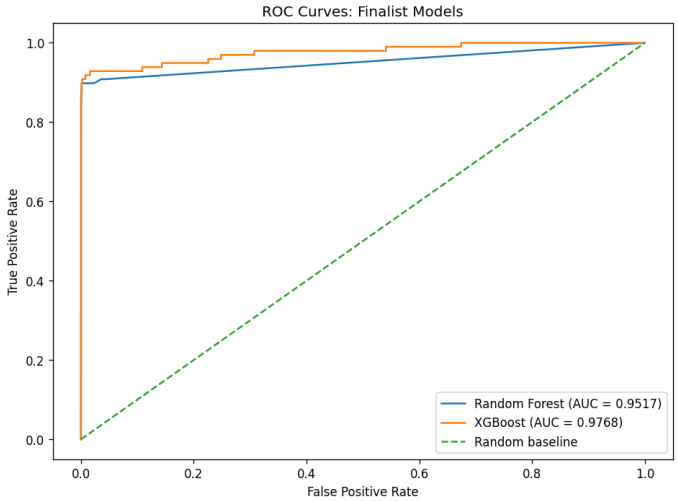


Figure 6: ROC curves for the finalist models.

Both Random Forest and XGBoost achieved ROC curves far above the random baseline, confirming that both models are highly effective at separating fraudulent from legitimate transactions. However, XGBoost achieved the higher **ROC-AUC**, which indicates a somewhat stronger overall ability to distinguish the two classes. This does not mean that ROC analysis should replace Precision–Recall analysis in this problem. Because fraud is rare, Precision–Recall remains especially important. Even so, the ROC curve provides useful additional evidence that XGBoost offers the stronger threshold-independent discrimination of the two finalists.

7 Feature Importance and Interpretability

In addition to predictive performance, a final model-selection decision should also consider interpretability. Although neither Random Forest nor XGBoost is as directly interpretable as Logistic Regression or a small Decision Tree, both finalists still provide useful forms of model understanding through feature-importance analysis.

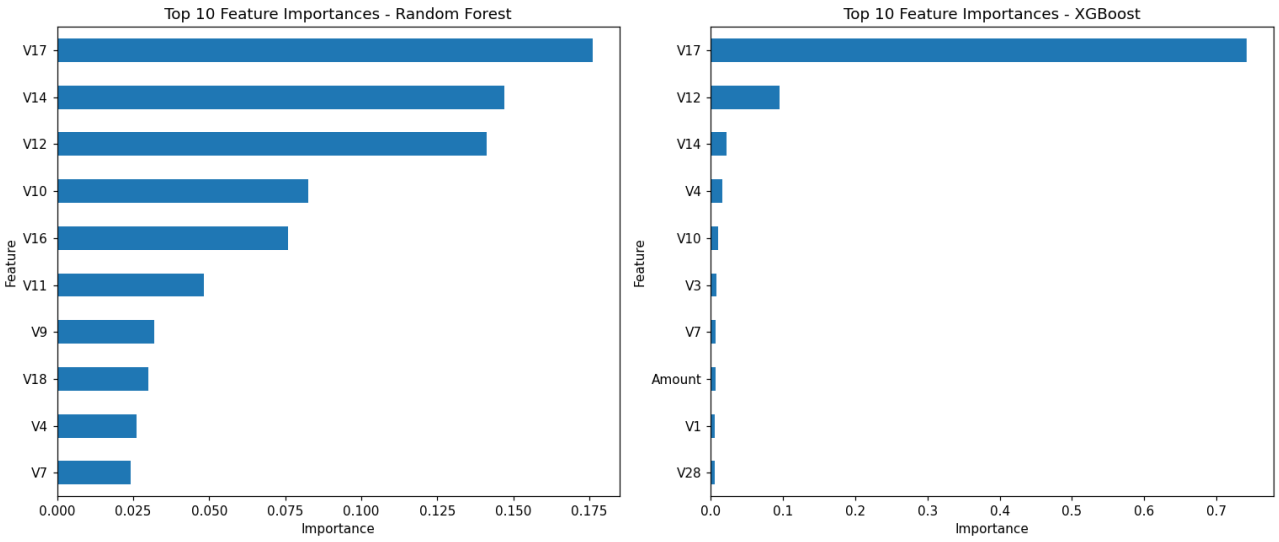


Figure 7: Top feature importances for the finalist models.

The feature-importance plots show that both models relied strongly on a relatively small group of variables, with substantial importance assigned to features such as `V17`, `V14`, `V12`, `V10`, `V11`, and `V4`. This is consistent with the earlier exploratory analysis, where several of these features already appeared to have stronger relationships with the fraud class. This consistency is useful because it increases confidence that the finalists are learning meaningful structure rather than arbitrary noise. At the same time, the two models use the feature space somewhat differently. Random Forest distributes importance more broadly across several predictors, which suggests a more balanced use of multiple variables. XGBoost appears to rely more strongly on a smaller set of highly informative variables. This does not mean one model is more correct than the other; it simply shows that the two models capture useful fraud-related structure in different ways.

Feature importance should still be interpreted with caution. Importance values indicate how much a model used a feature during prediction, but they do not prove that the feature has a direct causal relationship with fraud. Even so, they remain a useful interpretability tool for understanding what the final classifiers are relying on.

7.1 Illustrative tree visualisation

To support interpretability further, one representative tree from the Random Forest ensemble was also visualised. This does not represent the full model, since Random Forest makes predictions through the combined output of many trees, but it is still helpful because it shows the type of decision rules used inside the ensemble.

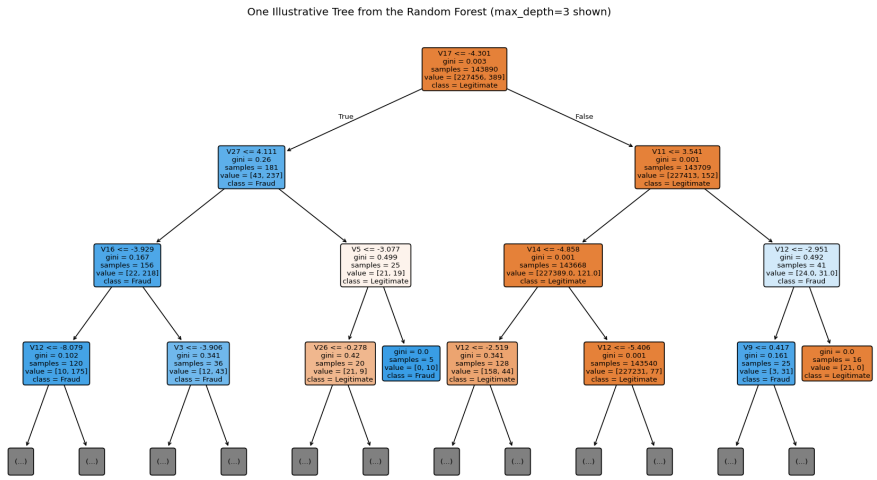


Figure 8: One illustrative tree extracted from the Random Forest ensemble.

The illustrative tree confirms that the model relies on splits involving variables such as `V17`, `V14`, `V12`, `V11`, and `V16`, which is broadly consistent with both the feature-importance results and the earlier exploratory analysis. Its role is explanatory rather than definitive: it helps interpret the model structure, but it should not be mistaken for the Random Forest model as a whole.

8 Final Model Selection Based on Business Requirements

The final choice between Random Forest and XGBoost depends on the business trade-off that the institution wishes to prioritise:

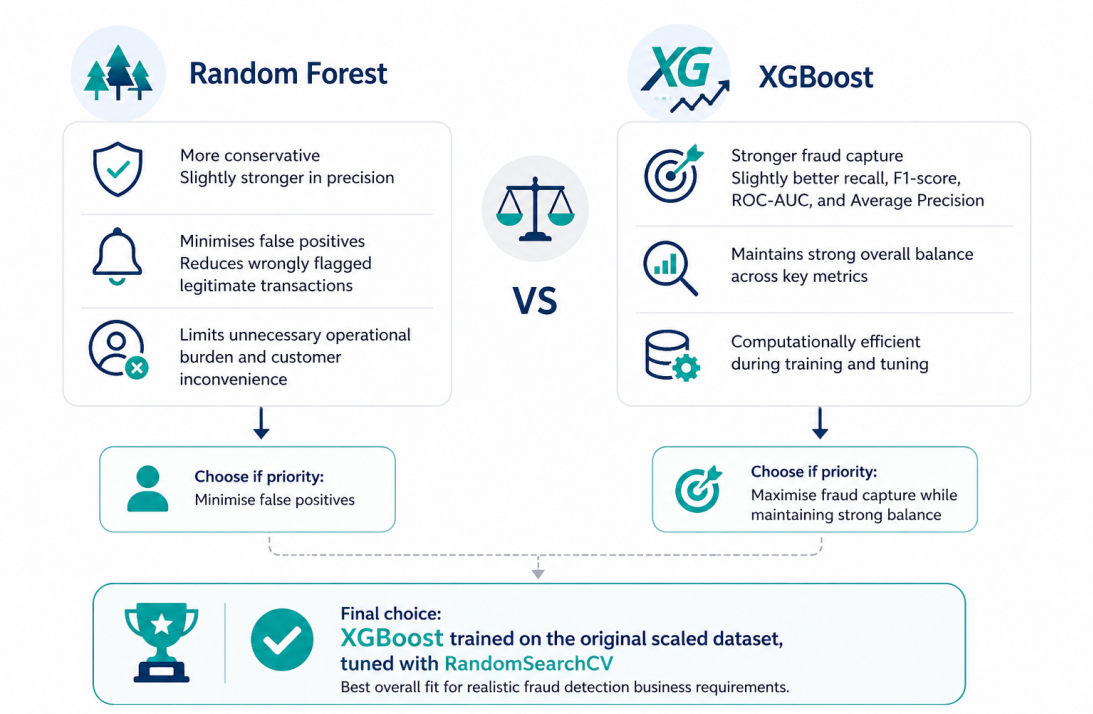


Figure 9: Comparison of final models and final choice.

If the main concern is to minimise false positives as much as possible, **Random Forest** remains highly attractive because it is slightly more conservative and slightly stronger in precision. This would reduce the number of legitimate transactions incorrectly flagged as fraud and therefore limit unnecessary operational burden and customer inconvenience. However, if the main concern is to maximise fraud capture while still maintaining strong overall balance, **XGBoost** is the stronger final choice. Across the final comparison, XGBoost achieved slightly better recall, F1-score, ROC-AUC, and Average Precision, while remaining highly competitive in precision. Since missed fraud cases are especially costly in financial settings, this gives XGBoost the stronger overall business case. A further practical advantage is that XGBoost also remained computationally efficient during the training and tuning stages, whereas Random Forest was somewhat more expensive to validate and tune repeatedly. This does not override predictive performance, but it strengthens the case for XGBoost as the more operationally attractive final recommendation.

9 Conclusion

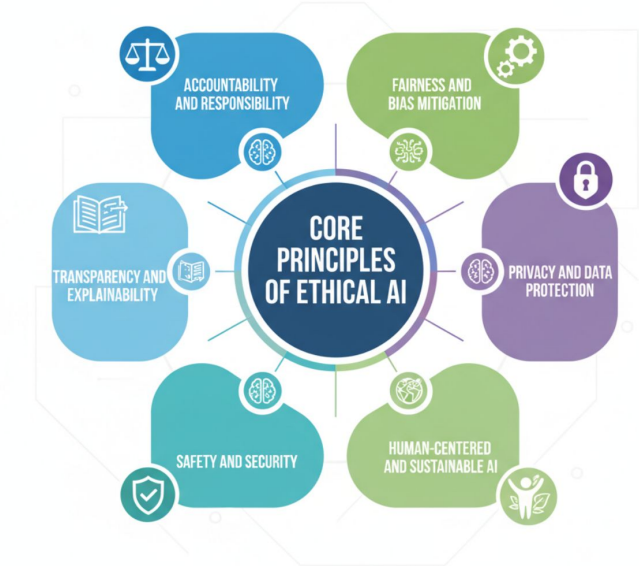
The model-evaluation stage showed that both Random Forest and XGBoost are strong fraud-detection models. Random Forest offered slightly better precision, but XGBoost achieved better recall, F1-score, ROC-AUC and Average Precision. Because missed fraud is especially costly, XGBoost provides the stronger final business balance. The final recommendation is therefore **XGBoost trained on the original scaled dataset, tuned with RandomizedSearchCV**.

Figure 10: Recommended final model configuration.

It combines strong fraud capture, high overall classification quality, competitive interpretability through feature importance, and good computational practicality. The model-evaluation stage therefore completes the model-selection process by identifying not only a strong classifier, but the one that best fits the business requirements of realistic fraud detection.

Technical Portfolio Report

Responsible AI, Ethics, and Governance



Section Contents

1 Responsible AI, Ethics, and Governance **49**
1.1 Bias and Fairness 49
1.2 Transparency and Explainability 49
1.3 Data Privacy and Cloud Governance 49
1.4 Professional Responsibility in Fraud Detection 49
1.5 Sustainability and Responsible Cloud Use 50

2 Conclusion **50**

1 Responsible AI, Ethics, and Governance

Fraud detection can create clear business value, but its deployment on public cloud platforms also raises important ethical and professional concerns. These systems work with sensitive financial and behavioural data, so poor design can lead to unfair outcomes, privacy weaknesses, limited transparency, and irresponsible operational decisions.

1.1 Bias and Fairness

One major ethical issue is **bias and fairness**. Fraud datasets are not neutral records of reality. They are influenced by past reporting practices, investigation priorities, institutional rules, and existing operational thresholds. As a result, a model may learn patterns that disadvantage certain customers or transaction types, even when protected characteristics are not directly included (Barocas and Selbst, 2016; Mehrabi et al., 2021).

In practice, this can mean that some users are flagged more often than others without a strong justification. In financial services, this is ethically serious because false positives can block legitimate transactions, delay access to money, and damage customer trust. Professional fraud detection therefore requires more than strong predictive accuracy. It also requires regular attention to how errors are distributed and whether the system may be repeating historical bias in an indirect way.

1.2 Transparency and Explainability

A second issue is **transparency and explainability**. Fraud detection models are often used in contexts where their outputs can lead to further investigation, customer friction, or operational action. Organisations should therefore be able to explain, at least at a practical level, why a transaction was flagged and how the model is controlled (Guidotti et al., 2018).

This does not mean that every model must be fully interpretable in mathematical terms. It does mean that the system should be documented, auditable, and clear enough for internal teams, regulators, and compliance staff. In professional practice, this includes model documentation, decision logging, drift monitoring, and evidence that the model remains suitable over time.

1.3 Data Privacy and Cloud Governance

A third concern is **data privacy and cloud governance**. Fraud detection depends on highly sensitive financial and behavioural information, so cloud deployment needs careful professional management. Key issues include lawful processing, data minimisation, purpose limitation, access control, encryption, retention policies, and secure handling of personal data (European Union, 2016; Information Commissioner's Office, 2020).

Cloud platforms provide useful security and compliance tools, but they do not remove the responsibility of the organisation using them. Under a shared responsibility model, the institution remains accountable for how data is collected, stored, accessed, and used. Professional practice therefore requires clear governance over permissions, monitoring, audit trails, and cross-border data handling, especially when the system is deployed at scale.

1.4 Professional Responsibility in Fraud Detection

Professional responsibility also involves careful management of the **false positive and false negative trade-off**:

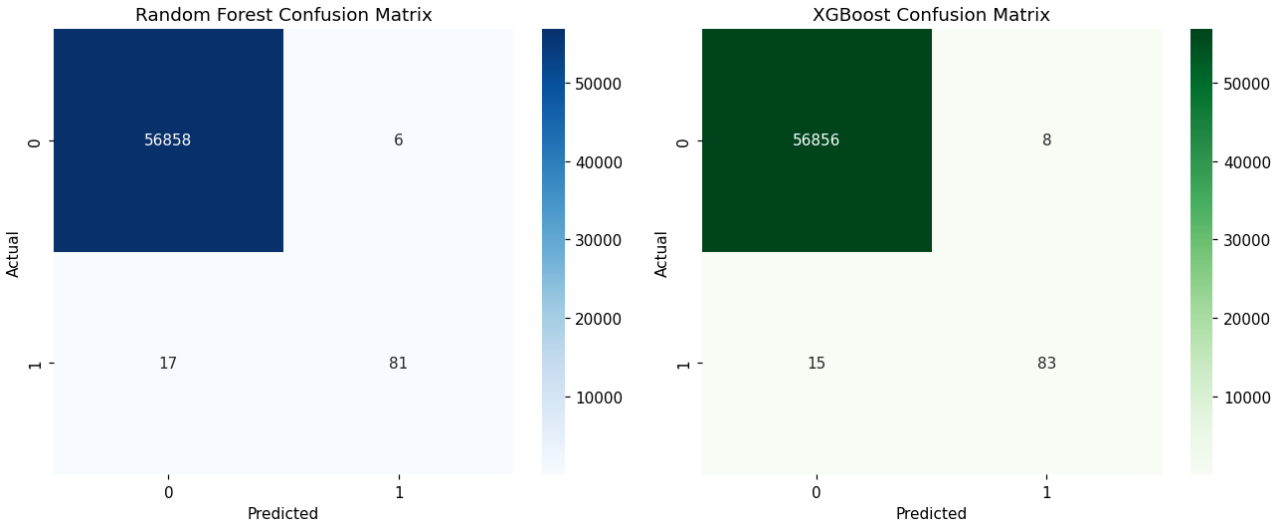


Figure 1: False-positive and false-negative trade-off for the finalist models.

In fraud detection, missed fraud can be costly, but flagging too many legitimate transactions can also harm customers through unnecessary reviews, blocked payments, or loss of trust. This trade-off shows why a professionally responsible team should not present the model as fully objective or automatically correct. Responsible deployment requires threshold review, human oversight, monitoring, periodic revalidation, and clear communication about model uncertainty, limitations, and possible harmful errors (Dal Pozzolo et al., 2018).

1.5 Sustainability and Responsible Cloud Use

A final issue is **sustainability**. Cloud-based AI systems consume energy during training, tuning, storage, and deployment. This footprint increases when models are retrained unnecessarily or when large computational searches are used without meaningful performance gains (Strubell, Ganesh and McCallum, 2019; Schwartz et al., 2020).

Sustainability is therefore both an environmental and professional concern. Responsible AI practice should include efficient experimentation, proportionate hyperparameter tuning, and avoidance of unnecessary computational cost. In this project, this principle was reflected in the decision not to continue with more expensive search strategies when they did not produce meaningful improvement.

Strategic Response

A professional and ethical fraud detection system must do more than achieve strong predictive performance. It also needs to be fair, privacy-aware, explainable, governable, and responsible in its use of computing resources. In cloud deployment, these requirements become especially important because scale, automation, and third-party infrastructure can increase both the benefits and the risks of AI-based decision-making.

2 Conclusion

Cloud-based fraud detection must be accurate, but it also needs to be fair, explainable, privacy-aware, auditable, and proportionate. Responsible deployment depends on strong data governance, careful management of model errors, human oversight, and awareness of computational cost.

Overall Conclusion

The dataset provides a strong basis for supervised fraud modelling because it is large, labelled, fully numerical, and free from missing values. At the same time, it presents two important constraints. Fraud accounts for only approximately 0.17% of transactions, so overall accuracy can be misleading, and the anonymised `V1-V28` features restrict direct operational interpretation. The analytical workflow therefore needed to combine robust predictive evaluation with careful attention to class imbalance, data leakage, interpretability, and governance.

Exploratory analysis confirmed the extreme target imbalance, substantial right-skew and outlier behaviour in `Amount`, and a limited set of features with comparatively stronger relationships to fraud. These findings informed a leakage-controlled preprocessing pipeline. No imputation or categorical encoding was required; IQR-based outlier filtering was used diagnostically but not retained in the final modelling data; the train–test split was stratified; and scaling was fitted only on the training partition. `StandardScaler` provided a consistent basis for comparing scale-sensitive and tree-based models. Random under-sampling, random over-sampling, SMOTE, and class-weighted learning were all assessed, but none was assumed to be beneficial by default.

The model-development process compared linear, tree-based, distance-based, boosting, and neural-network approaches. Broad screening was followed by five-fold stratified cross-validation, RandomizedSearchCV tuning, and supplementary threshold analysis. The results showed that resampling often increased fraud sensitivity at the expense of precision and overall balance. The strongest configurations instead used the original scaled data. Random Forest and XGBoost emerged as the most defensible finalists, while Logistic Regression, Decision Tree, KNN, and the neural-network experiments remained useful baselines and methodological comparisons.

Final evaluation confirmed that both finalists were highly capable but reflected a meaningful operational trade-off. Random Forest was slightly stronger in precision and produced slightly fewer false positives, making it attractive where alert reliability and reduced customer friction are prioritised. XGBoost achieved slightly stronger recall, F1-score, ROC-AUC, and Average Precision, indicating a better overall balance for capturing rare fraud. The supplementary threshold experiment did not justify replacing the default threshold of 0.5. On the evidence available, the final recommendation is therefore **XGBoost trained on the original scaled dataset, tuned with RandomizedSearchCV, and used with the retained default threshold**.

This recommendation should not be interpreted as a claim that model output is automatically correct or sufficient for autonomous decision-making. The anonymised data limits causal and business interpretation, historical fraud labels may reflect earlier investigation practices, and model performance can change as transaction behaviour evolves. Responsible use therefore requires human oversight, documented decision thresholds, audit trails, drift monitoring, periodic revalidation, privacy and access controls, and review of how false positives and false negatives affect customers and operational teams. Feature-importance evidence can support investigation and governance, but it does not remove the need for domain expertise or individual-case review.

Appendix A provides deployment context rather than altering the modelling conclusion. Its comparison of AWS, Azure, and GCP identifies Azure as the best overall fit for the regulated financial-services scenario considered, particularly because of its governance, compliance, enterprise-integration, cost-management, and machine-learning lifecycle capabilities. Any production implementation should nevertheless manage vendor lock-in, maintain portable documentation and metadata, and preserve the monitoring and human-control requirements established in the responsible-AI analysis.

References

Alkhatib, A., Shaheen, A. and Albustanji, R.N. (2025) 'A comparative analysis of cloud computing services: AWS, Azure, and GCP', *International Journal of Computing and Digital Systems*, 18(1), pp. 1–15. Available at: https://doi.org/10.12785/ijcds/1571111846 (Accessed: 10 May 2026).

Amazon Web Services (n.d.) 'Compliance validation for Amazon SageMaker AI'. Available at: https://docs.aws.amazon.com/sagemaker/latest/dg/sagemaker-compliance.html (Accessed: 10 May 2026).

Amazon Web Services (2025) 'Introducing the AWS User Guide to Governance, Risk and Compliance for Responsible AI Adoption within Financial Services Industries'. Available at: https://aws.amazon.com/blogs/security/introducing-the-aws-user-guide-to-governance-risk-and-compliance-for-responsible-ai-adoption-within-financial-services-industries/ (Accessed: 10 May 2026).

Barocas, S. and Selbst, A.D. (2016) 'Big Data's Disparate Impact', *California Law Review*, 104(3), pp. 671–732.

Breiman, L. (2001) 'Random Forests', *Machine Learning*, 45(1), pp. 5–32. Available at: https://doi.org/10.1023/A:1010933404324 (Accessed: 10 May 2026).

Carcillo, F., Dal Pozzolo, A., Le Borgne, Y.-A., Caelen, O., Mazzer, Y. and Bontempi, G. (2018) 'SCARFF: A scalable framework for streaming credit card fraud detection with Spark', *Information Fusion*, 41, pp. 182–194. Available at: https://www.sciencedirect.com/science/article/abs/pii/S1566253517305444 (Accessed: 10 May 2026).

Chawla, N.V., Bowyer, K.W., Hall, L.O. and Kegelmeyer, W.P. (2002) 'SMOTE: Synthetic Minority Oversampling Technique', *Journal of Artificial Intelligence Research*, 16, pp. 321–357. Available at: https://www.jair.org/index.php/jair/article/view/10302 (Accessed: 10 May 2026).

Chen, T. and Guestrin, C. (2016) 'XGBoost: A Scalable Tree Boosting System', in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 785–794. Available at: https://doi.org/10.1145/2939672.2939785 (Accessed: 10 May 2026).

Choi, W., Choi, T. and Heo, S. (2023) 'A comparative study of automated machine learning platforms for exercise anthropometry-based typology analysis: performance evaluation of AWS SageMaker, GCP VertexAI, and MS Azure', *Bioengineering*, 10(8), 891. Available at: https://doi.org/10.3390/bioengineering10080891 (Accessed: 10 May 2026).

Dal Pozzolo, A., Boracchi, G., Caelen, O., Alippi, C. and Bontempi, G. (2018) 'Credit Card Fraud Detection: A Realistic Modeling and a Novel Learning Strategy', *IEEE Transactions on Neural Networks and Learning Systems*, 29(8), pp. 3784–3797. Available at: https://pubmed.ncbi.nlm.nih.gov/28920909/ (Accessed: 10 May 2026).

European Union (2016) 'Regulation (EU) 2016/679 of the European Parliament and of the Council (General Data Protection Regulation)', *Official Journal of the European Union*, L119, pp. 1–88.

FICO (2025) 'Understanding fraud detection to safeguard financial services in an ever-changing threat landscape', *FICO Blog*, 23 December. Available at: https://www.fico.com/blogs/understanding-fraud-detection-safeguard-financial-services-ever-changing-threat-landscape (Accessed: 10 May 2026).

Google Cloud (2025) 'Security, privacy, and compliance'. Available at: https://docs.cloud.google.com/architecture/framework/perspectives/fsi/security (Accessed: 10 May 2026).

Google Cloud (2026) 'Google Cloud for financial services'. Available at: https://cloud.google.com/solutions/financial-services (Accessed: 10 May 2026).

Guidotti, R., Monreale, A., Ruggieri, S., Turini, F., Giannotti, F. and Pedreschi, D. (2018) 'A Survey of Methods for Explaining Black Box Models', *ACM Computing Surveys*, 51(5), pp. 1–42.

He, H. and Garcia, E.A. (2009) 'Learning from imbalanced data', *IEEE Transactions on Knowledge and Data Engineering*, 21(9), pp. 1263–1284.

Information Commissioner's Office (2020) 'Guidance on AI and data protection'. Available at: https://ico.org.uk/for-organisations/uk-gdpr-guidance-and-resources/artificial-intelligence/ (Accessed: 10 May 2026).

Mehrabi, N., Morstatter, F., Saxena, N., Lerman, K. and Galstyan, A. (2021) 'A Survey on Bias and Fairness in Machine Learning', *ACM Computing Surveys*, 54(6), pp. 1–35.

Microsoft (2023a) 'FCA and PRA (UK) - Azure Compliance'. Available at: https://learn.microsoft.com/en-us/azure/compliance/offerings/offering-fca-pra-uk (Accessed: 10 May 2026).

Microsoft (2023b) 'Azure and other Microsoft cloud services compliance offerings'. Available at: https://learn.microsoft.com/en-us/azure/compliance/offerings/ (Accessed: 10 May 2026).

Microsoft (2025a) 'Responsible AI in Azure Workloads'. Available at: https://learn.microsoft.com/en-us/azure/well-architected/ai/responsible-ai (Accessed: 10 May 2026).

Microsoft (2025b) 'What is Azure Machine Learning?'. Available at: https://learn.microsoft.com/en-us/azure/machine-learning/overview-what-is-azure-machine-learning?view=azureml-api-2 (Accessed: 10 May 2026).

Microsoft (2025c) 'MLOps model management with Azure Machine Learning'. Available at: https://learn.microsoft.com/en-us/azure/machine-learning/concept-model-management-and-deployment?view=azureml-api-2 (Accessed: 10 May 2026).

Microsoft (2025d) 'Overview of Cost Management'. Available at: https://learn.microsoft.com/en-us/azure/cost-management-billing/costs/overview-cost-management (Accessed: 10 May 2026).

Microsoft (2025e) 'Optimize your cloud investment with Cost Management'. Available at: https://learn.microsoft.com/en-us/azure/cost-management-billing/costs/cost-mgt-best-practices (Accessed: 10 May 2026).

Saito, T. and Rehmsmeier, M. (2015) 'The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets', *PLOS ONE*, 10(3), e0118432. Available at: https://doi.org/10.1371/journal.pone.0118432 (Accessed: 10 May 2026).

Schwartz, R., Dodge, J., Smith, N.A. and Etzioni, O. (2020) 'Green AI', *Communications of the ACM*, 63(12), pp. 54–63.

scikit-learn developers (2026a) 'confusion_matrix'. Available at: https://scikit-learn.org/stable/modules/generated/sklearn.metrics.confusion_matrix.html (Accessed: 10 May 2026).

scikit-learn developers (2026b) 'Tuning the hyper-parameters of an estimator'. Available at: https://scikit-learn.org/stable/modules/grid_search.html (Accessed: 10 May 2026).

scikit-learn developers (2026c) 'Model evaluation: classification metrics'. Available at: https://scikit-learn.org/stable/modules/model_evaluation.html (Accessed: 10 May 2026).

scikit-learn developers (2026d) 'Pipeline'. Available at: https://scikit-learn.org/stable/modules/generated/sklearn.pipeline.Pipeline.html (Accessed: 10 May 2026).

scikit-learn developers (2026e) 'Common pitfalls and recommended practices'. Available at: https://scikit-learn.org/stable/common_pitfalls.html (Accessed: 10 May 2026).

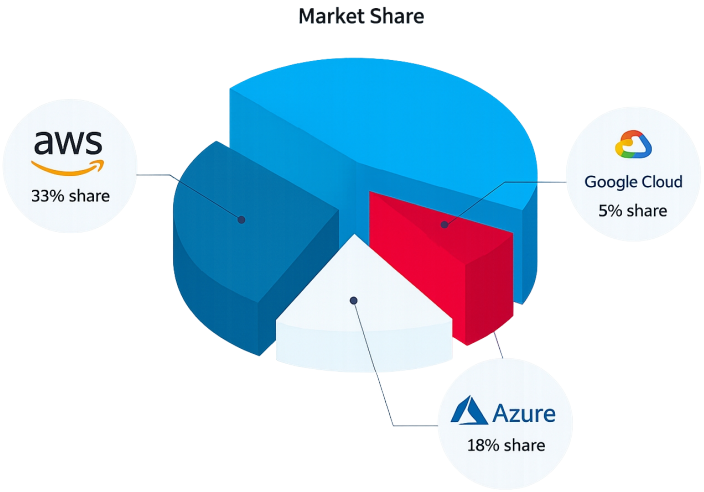
scikit-learn developers (2026f) 'Preprocessing data'. Available at: https://scikit-learn.org/stable/modules/preprocessing.html (Accessed: 10 May 2026).

scikit-learn developers (2026g) 'RandomizedSearchCV'. Available at: https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.RandomizedSearchCV.html (Accessed: 10 May 2026).

Strubell, E., Ganesh, A. and McCallum, A. (2019) 'Energy and Policy Considerations for Deep Learning in NLP', in *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pp. 3645–3650.

Verma, A. (2025) 'A comparative analysis of cloud providers for scalable and reliable systems', *World Journal of Advanced Engineering Technology and Sciences*, 15(2), pp. 2520–2529. Available at: https://doi.org/10.30574/wjaets.2025.15.2.0809 (Accessed: 10 May 2026).

A Cloud Deployment Feasibility



A.1 Introduction

This appendix evaluates the feasibility of deploying a fraud-detection machine-learning solution on a public-cloud platform. The scenario is a financial-services organisation seeking effective, compliant, scalable fraud detection. A cloud feasibility study is necessary because, in a regulated financial environment, the "best" platform is not simply the fastest or the cheapest. A suitable platform must also support:

Figure 1: Cloud-platform requirements for regulated fraud detection.

For this reason, three major cloud providers are compared in this appendix: **Amazon Web Services (AWS)**, **Microsoft Azure (Azure)**, and **Google Cloud Platform (GCP)**. The comparison is made specifically in relation to a **production-oriented fraud-detection workflow** rather than a general-purpose cloud adoption decision.

A.2 Evaluation Framework

To make the comparison consistent, the three platforms are evaluated against six criteria that are especially relevant for cloud deployment in financial services:

Figure 2: Evaluation criteria used for the cloud-platform comparison.

These criteria are not equally important in every industry. For example, a consumer start-up might emphasise rapid experimentation and lowest-cost compute, whereas a financial institution must place much greater weight on compliance, model governance, transparency, operational control, and auditability. This distinction is important because the machine-learning workload in this project is not trivial. The analytical and modelling workflow involves class imbalance handling, model comparison, cross-validation, hyperparameter tuning, and final model selection. A suitable cloud platform must therefore support not only notebook experimentation but also reproducible model lifecycle management.

Key Insight

For a fraud-detection use case in financial services, **compliance, governance, and integration** are not secondary considerations. They are part of the core feasibility question.

A.3 Comparative Analysis of AWS, Azure, and GCP

A.3.1 Performance

All three hyperscalers provide strong machine-learning performance, but "performance" must be interpreted carefully. In an academic or experimental benchmark, performance may refer mainly to training speed or predictive accuracy. In a financial-services context, however, useful performance also includes:

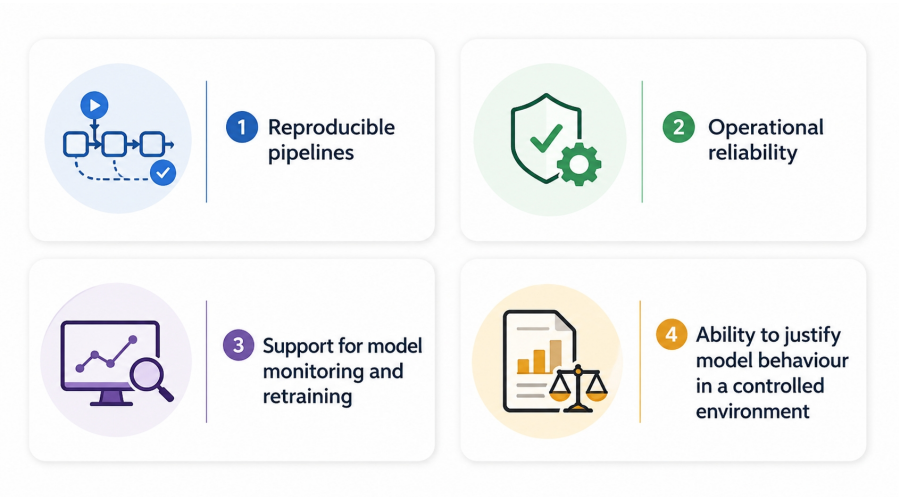


Figure 3: Cloud-performance dimensions for fraud-detection deployment.

AWS is often considered the benchmark leader in raw breadth of cloud and machine-learning services. SageMaker offers strong training, deployment, registry, and monitoring capabilities, and AWS has also published financial-services-specific guidance for responsible AI governance and risk management. This makes AWS a very strong technical candidate (Alkhatib, Shaheen and Albustanji, 2025; Choi, Choi and Heo, 2023; Amazon Web Services, 2025, n.d.). GCP is also highly capable, especially through Vertex AI. Google presents Vertex AI as a unified AI platform for financial services, supported by a security and compliance framework and strong AI tooling. This makes GCP particularly attractive for organisations already invested in Google's AI ecosystem (Google Cloud, 2026, 2025; Choi, Choi and Heo, 2023). Azure's case is slightly different. It may not always dominate in generic platform-performance narratives, but it performs strongly where regulated enterprises often care most: **governed ML lifecycle management**. Azure Machine Learning is explicitly designed to manage the machine-learning project lifecycle, including training, deployment, monitoring, retraining, and reproducible pipelines. In addition, Azure Workloads guidance explicitly connects AI system design with responsible AI, privacy, explainability, and auditability (Microsoft, 2025b,c,a).

For fraud detection, this broader view of performance matters more than isolated benchmark results. A platform that supports reproducibility, monitoring, and controlled deployment is more valuable than one focused mainly on experimentation.

A.3.2 Scalability

Scalability is essential because financial fraud systems may need to support:

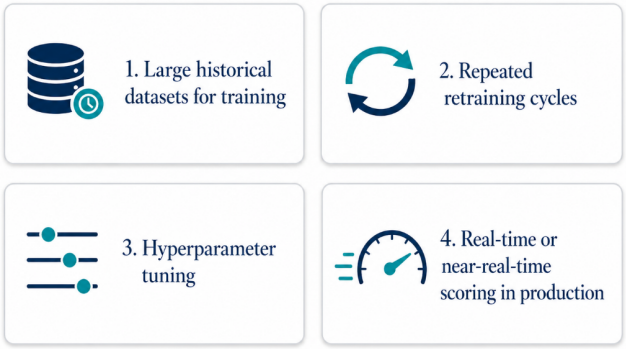


Figure 4: Scalability requirements for fraud-detection deployment.

AWS is particularly strong in large-scale infrastructure control and is widely recognised for mature scaling options in machine-learning environments (Alkhatib, Shaheen and Albustanji, 2025; Verma, 2025). GCP also scales well and benefits from a strong AI-platform narrative, especially when organisations want a tightly integrated AI development environment around Vertex AI and related services (Google Cloud, 2026). Azure remains highly competitive here, but its main advantage in this use case is that its scalability is tied closely to **enterprise-operational manageability**. Azure's cloud-adoption and MLOps guidance supports an incremental path from experimentation to mature MLOps practice (Microsoft, 2025c,b). For a regulated institution, this can be more important than simply maximising raw scale, because scaling must remain governable, auditable, and maintainable. From that perspective, Azure's scalability is not just about infrastructure expansion. It is about scaling a machine-learning system in a way that remains organisationally usable and compliant.

A.3.3 Cost

Cost feasibility in cloud adoption is not only about the nominal price of compute. In practice, total cost is affected by:

Figure 5: Cost-governance factors in the cloud assessment.

All three providers use usage-based pricing models, but they differ in how well they support enterprise cost control. AWS offers broad flexibility, but that flexibility can also make cost estimation and optimisation more complex. For some organisations, this is acceptable; for regulated financial institutions with tight budget controls and governance expectations, it can become an operational burden. GCP is often perceived as relatively clear in pricing structure, with strong data and AI tooling. However, clarity of pricing alone does not automatically make it the strongest option when wider enterprise budgeting and governance processes are considered. Azure is particularly strong in this area because Microsoft Cost Management provides tooling for planning, analysing, forecasting, and monitoring cloud spend. Azure documentation also emphasises cost analysis, budgeting, and reporting as ongoing governance mechanisms rather than one-off setup activities (Microsoft, 2025d,e). This matters in a financial-services setting where cost predictability is itself part of good operational governance. A platform that makes cost behaviour easier to track and justify is more feasible in practice than one that is technically powerful but financially harder to control.

A.3.4 Compliance and governance

This is arguably the most important criterion in this feasibility analysis.

Financial institutions operate in a highly regulated environment, where cloud adoption is shaped by legal, regulatory, and governance constraints. Relevant concerns include:

Figure 6: Compliance and governance factors in the cloud assessment.

AWS is strong in governance and compliance. It provides financial-services-specific guidance on governance, risk, and compliance for responsible AI adoption, and SageMaker supports compliance-aware ML workflows (Amazon Web Services, 2025, n.d.). GCP also presents a serious offer for financial services, including security, compliance, sovereign cloud positioning, and Vertex AI support for governed AI development (Google Cloud, 2026, 2025). However, Azure is particularly well aligned to this use case because Microsoft has published explicit guidance for UK financial-services firms regulated by the FCA and PRA. Azure compliance offerings are also clearly documented and organised across globally applicable, industry-specific, and region-specific assurance categories. In addition, Microsoft's Well-Architected guidance for Responsible AI directly addresses issues such as privacy, transparency, user data handling, governance policies, and auditability (Microsoft, 2023a,b, 2025a). This gives Azure a particularly strong position for an institution that must justify cloud adoption not only technically but also procedurally and regulatorily.

Challenge

Cloud feasibility in financial services is not simply a matter of "can it run the model?" It is also a matter of whether the organisation can defend, govern, audit, and sustain the system over time. On this criterion, Azure is especially strong.

A.3.5 Integration with enterprise and ML workflows

Fraud-detection systems are rarely isolated experiments. They typically need to integrate with:

Figure 7: Integration points for enterprise fraud-detection workflows.

AWS provides a very broad ecosystem and integrates well across data and machine-learning workflows. This is a major strength, especially for cloud-native organisations already operating heavily in AWS. GCP also offers strong AI-centric integration, especially where organisations already use Google data or analytics services. Azure's distinctive strength is that it often fits naturally into enterprise environments that already use Microsoft's broader ecosystem for identity, security, productivity, and governance. Azure Machine Learning workspaces also support collaboration, run-history tracking, logs, metrics, lineage metadata, and model lifecycle control, all of which are valuable for a governed fraud-detection workflow (Microsoft, 2025b,c).
This is especially relevant to this project, because the fraud-detection pipeline includes experimentation, model comparison, resampling strategies, cross-validation, tuning, and final model selection. A production cloud platform must be able to support that workflow in a controlled and reproducible manner.

A.3.6 Vendor lock-in

Vendor lock-in is a genuine strategic risk across all three providers (Alkhatib, Shaheen and Albustanji, 2025; Verma, 2025). Machine-learning platforms increasingly encourage the use of provider-specific services for training, deployment, monitoring, governance, and AI tooling. The more tightly an organisation adopts these services, the harder it can become to migrate later.
AWS, Azure, and GCP all create some degree of lock-in through:

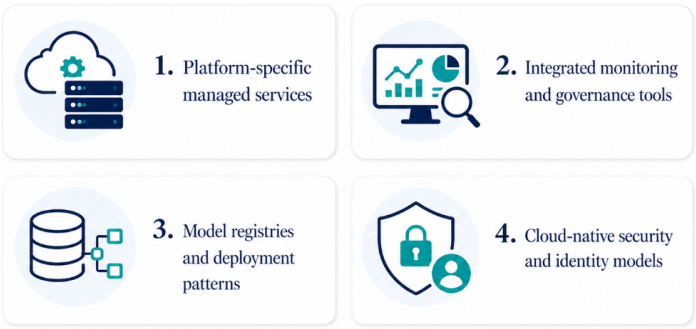


Figure 8: Vendor lock-in mechanisms in cloud ML platforms.

Therefore, vendor lock-in is not a reason to reject one provider and embrace another uncritically. Instead, it should be treated as a **risk to manage**. In practice, the most realistic question is whether the operational and governance benefits of a platform outweigh the migration constraints it may later create. For this project, the answer is yes in the case of Azure. The governance, compliance, and enterprise-integration benefits are

sufficiently strong that the lock-in risk appears acceptable, provided the organisation applies good architectural discipline, such as reproducible pipelines, version control, model documentation, and clear export strategies for data and metadata where possible.

Azure's advantages also involve a strategic trade-off: deeper reliance on Azure Machine Learning, Microsoft compliance tooling, cost-management services, and Microsoft-centred governance may make future migration harder. This does not weaken the recommendation, but it means lock-in should be managed deliberately rather than treated as an afterthought.

A.4 Comparative Summary

Criterion	AWS	Azure	GCP
Performance	Very strong technical breadth and mature ML tooling; strong infrastructure and governance options.	Strong lifecycle-oriented ML performance with responsible-AI and MLOps support.	Strong unified AI platform through Vertex AI; very capable AI ecosystem.
Scalability	Excellent infrastructure scale and service breadth.	Strong enterprise-manageable scalability with MLOps maturity guidance.	Strong platform scalability, especially for AI-centric environments.
Cost	Flexible but can be harder to forecast and govern at enterprise level.	Strong cost-management, reporting, budgeting, and planning support.	Clear and competitive in some scenarios, but less compelling here on governance grounds.
Compliance / governance	Strong and credible for financial services.	Especially strong for UK-regulated financial firms; strong responsible-AI and compliance posture.	Strong security and compliance positioning, including financial-services framing.
Integration	Broad ecosystem; excellent for AWS-heavy organisations.	Very strong fit for enterprise governance and Microsoft-centred environments.	Strong for Google AI/data ecosystems.
Vendor lock-in	Present	Present	Present
Overall suitability for this use case	Very strong contender	**Best overall fit**	Strong but less compelling than Azure for this specific regulated scenario

Table 1: AWS, Azure, and GCP comparison for regulated fraud detection.

A.5 Recommendation

Based on the comparison, **Microsoft Azure** is recommended as the most suitable cloud platform for this fraud-detection use case.

This does not mean that Azure is the best option in every situation. AWS remains very strong, especially in service breadth and infrastructure maturity, while GCP is a credible choice for AI-focused organisations.

However, for this specific scenario, a fraud-detection system in a regulated financial environment, Azure provides the best overall fit, particularly in terms of compliance, governance, enterprise integration, cost control, and ML lifecycle support.

The recommendation is based on five main reasons:

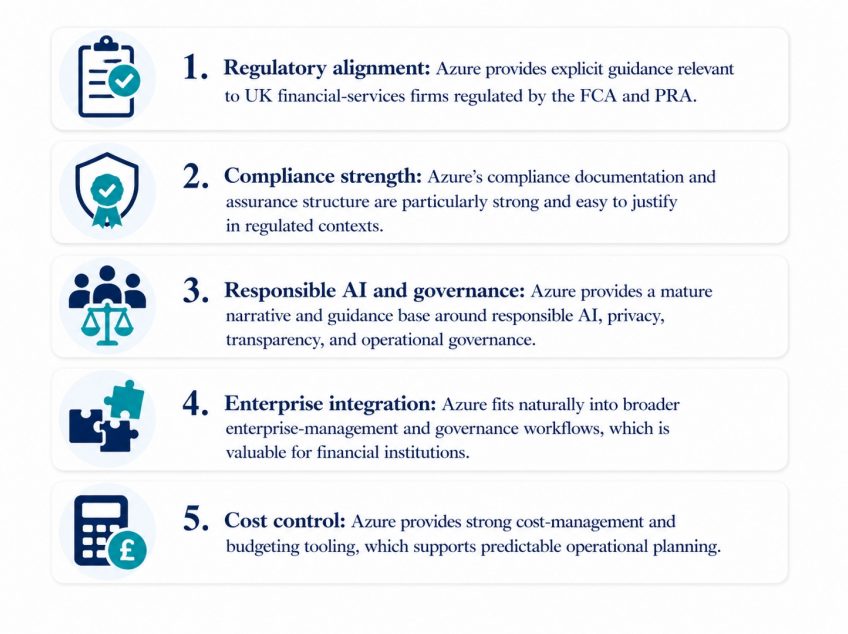


Figure 9: Azure strengths for regulated financial workflows.

Another important point is that the project itself was prototyped in a notebook-based academic environment, but a production recommendation should not be based on the prototyping environment used for academic experimentation. Instead, it should be based on the long-term requirements of deployment, governance, risk management, and organisational fit. On those criteria, Azure is the most suitable option (Microsoft, 2023a,b, 2025a,b,d).

Strategic Response

The strategic recommendation is therefore clear: **use Azure as the target production cloud platform for this fraud-detection solution**, while recognising AWS as the strongest alternative and GCP as a credible but less aligned option for this particular regulatory and enterprise context.

A.6 Conclusion

AWS, Azure, and GCP can all support the fraud-detection solution. However, feasibility depends on more than technical capability. It also requires strong governance, compliance, enterprise integration, cost control, and auditable ML lifecycle management.

AWS offers strong maturity and service breadth, while GCP provides advanced AI capabilities and a credible financial-services offering. However, Azure is the best fit for this project because it aligns most closely with the needs of a regulated fraud-detection system.

Therefore, Azure is recommended as the most feasible cloud platform for deploying the fraud-detection solution developed in this portfolio report.